



A Real-Time Music-Conditioned Framework for Motion Retrieval, Retargeting, and Adaptive Humanoid Control

Chengjun Ouyang¹

MSc Machine Learning

Supervisor: Prof Chengxu Zhou

Submission date: 07/08/2026

¹**Disclaimer:** This report is submitted as part requirement for the MSc Machine Learning at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged

Abstract

Live music-driven humanoid dance requires musical relevance, responsive timing, continuous motion transitions, and feasible robot configurations to be maintained under causal audio access and limited computation. This dissertation presents BeatWeaver, a retrieval-and-control system that adapts a finite library of dance trajectories to streaming music on a 29-degree-of-freedom Unitree G1 model.

The offline pipeline combines paired AIST++ music and human motion, extracts salient key poses from whole-body velocity minima, and retargets SMPL sequences through General Motion Retargeting. Provenance checks and MuJoCo kinematic preflight produce a catalogue of 411 admitted motions associated with 60 music identities and 348 audio segments. Online, six-second rolling audio windows are gain-normalised and encoded using Discogs EffNet embeddings, style activations, and rhythm–timbre descriptors. Hierarchical retrieval combines musical similarity with tempo, key-pose-density, and activity compatibility. Temporal evidence and bounded-hold selection govern motion changes, while asynchronous retrieval and candidate preparation support continuous playback. Motion-aware beat acceptance and bounded phase correction adapt timing gradually. Transition-entry search uses joint pose, velocity, foot contact, and root state, followed by root-aligned blending and joint-dynamics and collision constraints.

Strict leave-one-music-out evaluation achieves genre Recall@1 of 0.317 and Recall@5 of 0.783, indicating stronger shortlist relevance than top-one identification. In causal headless evaluation, all 135 formal runs, comprising 130 matched full-system and authored-timing short runs and five approximately ten-minute full-system runs, satisfy the registered timing, readiness, and final kinematic-output criteria at 60 Hz. The worst run-level control-work p99 is 14.483 ms, below the 16.67 ms budget, with no residual clearance or joint-range violations. A separate 120 Hz pilot passes only six of 21 runs. Harmonic beat alignment score relative to delivered online beat events is 0.313 with beat synchronisation and 0.306 with authored timing; the difference is not statistically significant after multiplicity correction.

The results establish an inspectable integration of music retrieval and continuous humanoid playback with an empirically validated 60 Hz operating point. Musical generalisation and the benefit of beat synchronisation remain limited or unconfirmed. Validation concerns kinematic playback in MuJoCo; dynamic balance, physical robot execution, and human-perceived dance quality require further evaluation.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Context and Gap	2
1.3	Problem Statement	3
1.4	Overview of the Proposed System	4
2	Literature Review	5
2.1	Introduction	5
2.2	Music Representation for Motion Control	5
2.2.1	Temporal and semantic information in music	5
2.2.2	Learned music embeddings	6
2.2.3	Beat and onset analysis under causal constraints	7
2.3	Dance Data and Motion Representation	8
2.3.1	Paired music–dance datasets	8
2.3.2	Human-body and robot-motion representations	9
2.4	Music-Conditioned Dance Generation	9
2.4.1	Autoregressive and discrete motion models	9
2.4.2	Diffusion-based dance synthesis	10
2.4.3	Retrieval within generative systems	11
2.5	Real-Time Audio-Driven Control	11
2.5.1	Causality, latency, and committed history	11
2.5.2	Direct audio-to-humanoid policies	11
2.5.3	Streaming diffusion forcing	12
2.6	Example-Based Motion Synthesis and Motion Matching	12
2.6.1	Motion graphs and database reuse	12
2.6.2	Motion matching and learned compression	13
2.7	Motion Transitions and Beat–Phase Synchronisation	13
2.7.1	State-compatible motion entry	13
2.7.2	Blending, root alignment, and output constraints	14
2.7.3	Beat-to-key-pose alignment	14
2.8	Humanoid Motion Retargeting and Validation	15
2.8.1	The embodiment gap	15
2.8.2	Simulation and the limits of kinematic preview	15

2.9	Critical Synthesis and Research Gap	16
3	Overall System Design	18
3.1	Chapter Overview	18
3.2	End-to-End Architecture	18
3.3	Subsystem Boundaries and Contracts	20
3.3.1	Offline preparation contract	20
3.3.2	Online perception and selection contract	20
3.3.3	Prepared switching and execution contract	21
3.4	State Ownership and Failure Isolation	21
4	Offline Data Preparation and Catalogue Construction	24
4.1	Chapter Overview	24
4.2	Source Data and Reproducibility Contracts	24
4.2.1	AIST++ pairing and source identity	24
4.2.2	Preparation of the SMPL body model	25
4.3	Audio Catalogue Construction	25
4.3.1	Windowing and robust signal preparation	25
4.3.2	Learned music representation	26
4.3.3	Rhythm, timbre, and activity descriptors	26
4.3.4	Dense array storage and normalisation statistics	27
4.4	Motion Key-Pose and Activity Analysis	27
4.4.1	Whole-body velocity signal	27
4.4.2	Valley detection and key-pose confidence	28
4.4.3	Motion profile and catalogue groups	29
4.5	SMPL-to-G1 Retargeting	29
4.5.1	Human forward kinematics and coordinate conversion	29
4.5.2	GMR inverse-kinematics solve	31
4.5.3	Collision-aware retargeting	31
4.5.4	Continuity conditioning	31
4.6	Canonical Trajectory and Dataset Build	32
4.6.1	Canonical G1 artefact	32
4.6.2	Cached builds, quarantine, and atomic publication	32
4.7	Trajectory Preflight and Catalogue Admission	33
4.7.1	MuJoCo kinematic preflight	33
4.7.2	Joining audio and motion records	33
4.8	Integrity Guarantees and Remaining Limitations	33
5	Real-Time Music–Motion Matching and Continuous Control	35
5.1	Chapter Overview	35
5.2	Online Problem Formulation	35
5.2.1	Dual-timescale causal processing	36
5.3	Hierarchical Music-to-Motion Retrieval	37
5.3.1	Query standardisation and segment similarity	37

5.3.2	Track pooling and genre-first restriction	37
5.3.3	Abstention and match confidence	38
5.3.4	Motion-specific compatibility	38
5.4	Stable Selection and Candidate Readiness	39
5.4.1	Relevance hysteresis	39
5.4.2	Diversity under a relevance constraint	40
5.4.3	Accepted-beat commitment and prepared candidates	40
5.5	Causal Beat and Phase Control	41
5.5.1	Activity gate and beat candidates	41
5.5.2	Motion-aware beat acceptance	41
5.5.3	Target rate and gradual phase correction	42
5.5.4	Bounded pose modulation	42
5.6	Transition-Entry Optimisation	43
5.6.1	Entry candidates and state features	43
5.6.2	Normalised entry cost	43
5.6.3	Feasibility and transition duration	44
5.7	Continuous Transition and Root Motion	44
5.7.1	Target-root alignment	44
5.7.2	Smooth pose and orientation blending	45
5.7.3	One-shot completion and state transfer	45
5.8	Output Conditioning	45
5.8.1	Stateful joint dynamics limits	45
5.8.2	Collision policy and state consistency	46
5.9	Method Observability and Failure Behaviour	46
6	Experimental Evaluation	47
6.1	Experimental Questions and Protocol	47
6.2	Datasets, Splits and Reproducibility	47
6.3	Evaluation Metrics	49
6.3.1	Retrieval, rejection and statistical units	49
6.3.2	Rhythm and motion diversity	49
6.3.3	Continuity, contact and constraints	50
6.4	Retrieval and Rejection Results	51
6.5	Rhythm, Diversity and Physical Plausibility	52
6.6	Transition Continuity and Safety	53
6.7	Inference Time and End-to-End Response	55
6.7.1	Computation, preparation and control work	55
6.7.2	Musical changes and observable response	56
6.8	Ablation Study	56
6.9	Long-Run Reliability	57
6.10	Discussion by Research Question	58

7	Conclusions and Future Work	60
7.1	Summary of the Work	60
7.2	Limitations	62
7.3	Future Work	63
A	Source Code Repository	69

Chapter 1

Introduction

1.1 Background and Motivation

Human movement is closely coupled to sound. Musical rhythm suggests when a movement should occur, tempo influences its speed, and timbre, dynamics, and phrasing shape its perceived character. Audio is therefore an attractive control modality for interactive characters and humanoid robots. It is temporally dense, naturally available in performance settings, and more expressive than a small set of discrete commands.

Music-driven motion is nevertheless a difficult computational problem. First, the relationship between music and dance is many-to-many, and compatibility depends on rhythm, timbre, activity, and style rather than tempo alone. A humanoid also imposes joint, collision, and support constraints that do not apply to an unconstrained animated skeleton. In a live setting, these issues must be addressed causally and within bounded latency while avoiding visible discontinuities when the music or selected movement changes.

Research has progressed from paired music-motion benchmarks and generative models such as AIST++/FACT and Bailando [18, 24] to diffusion-based choreography with improved diversity and control [1, 26, 17, 14]. More recent systems address limitations that are especially relevant to this dissertation: DiscoForcing formulates dance generation as a causal, bounded-latency streaming problem [15], while RoboPerform generates executable humanoid actions directly and avoids an explicit human-motion retargeting stage [19]. Together, these studies show the potential of learned music-conditioned motion, but they also emphasise the data, training, latency, and embodiment challenges involved in deploying a real-time humanoid system.

However, direct generative control is not the only useful design point. It requires substantial paired data, policy training, and a validated control stack. Where the available actions are known in advance, retrieval offers a complementary trade-off. Motions can be inspected, retargeted, and checked for the target robot before deployment, while selection and transition constraints remain explicit. Although this approach cannot create new choreography, it can provide predictable motion quality and modest runtime cost. This dissertation therefore investigates whether a retrieval-and-control pipeline can produce musically responsive, continuous humanoid dance from a finite, prevalidated motion library.

1.2 Research Context and Gap

The problem addressed in this dissertation lies at the intersection of three research areas: cross-modal music-motion matching, real-time motion scheduling, and humanoid motion retargeting. Each area is individually well studied, but their integration introduces unresolved engineering and research questions.

In cross-modal matching, a music representation must capture both semantic similarity and movement-relevant temporal structure. A representation based only on global style may retrieve a semantically related track while ignoring an incompatible tempo. Conversely, a representation based only on beats per minute may match two rhythmically similar tracks with very different energy or timbre. A motion also has its own temporal and expressive properties, including duration, characteristic speed, key-pose density, and the regularity of those key poses. Consequently, retrieving a similar catalogue track is not sufficient. The associated robot motion must also be feasible under the speed range of the online controller and appropriate for the activity of the query music.

In real-time scheduling, immediately selecting the highest-scoring motion at every retrieval step can lead to rapid oscillation among near-equal candidates. Requiring excessive evidence before switching, however, leaves the robot continuing an outdated movement after the music has changed. Even after a target motion has been selected, entering it at an arbitrary phase can produce joint, root, or contact discontinuities. A useful system must therefore separate four decisions: which music is most similar, which motion is most compatible, whether the evidence is stable enough to change motion, and where and when the new motion should begin.

In humanoid retargeting, human and robot embodiments differ in bone lengths, degrees of freedom, joint limits, and mechanical geometry. Retargeting errors such as foot sliding, ground penetration, self-intersection, and abrupt velocity spikes can degrade both perceptual fidelity and downstream control. The study accompanying General Motion Retargeting (GMR) demonstrates that the quality of the retargeted reference materially affects humanoid motion tracking performance [3]. This observation motivates an offline preflight stage in which candidate actions are transformed into the Unitree G1 joint space and rejected before they can enter the online catalogue if their metadata, joint order, continuity, ground contact, or collision properties are invalid.

The literature therefore leaves a practical gap between offline music-to-dance generation and a lightweight, inspectable real-time humanoid system. Specifically, there is value in a system that:

1. uses only current and past audio while maintaining a rolling musical context;
2. combines semantic music similarity with explicit rhythm, tempo, and motion compatibility;
3. suppresses unstable changes without eliminating musically acceptable diversity;
4. aligns musical events to salient poses without instantaneously teleporting the motion phase;
5. chooses transition entry states using pose, velocity, foot-contact, and root-motion information; and
6. outputs target-robot joint trajectories under explicit timing, kinematic, and collision constraints.

This dissertation addresses that gap through the design and evaluation of the *BeatWeaver*, a hybrid system comprising deterministic digital signal processing, optional learned audio embeddings, similarity-based retrieval, rule-governed temporal selection, and constrained kinematic motion control.

1.3 Problem Statement

Let a causal audio stream observed up to time t be denoted by $\mathcal{A}_{\leq t}$, and let the robot-ready motion library be

$$\mathcal{D} = \{(\mathbf{x}_i, M_i, \mathbf{m}_i)\}_{i=1}^N, \quad (1.1)$$

where $\mathbf{x}_i$ is the music descriptor associated with catalogue item i , M_i is a retargeted Unitree G1 trajectory, and $\mathbf{m}_i$ contains motion metadata such as original tempo, duration, activity, and salient key-pose phases. At runtime, the system must select a motion index i_t , a phase ϕ_t , and a continuous robot target configuration $\mathbf{q}_t$.

The objective is not to reproduce a unique ground-truth choreography. Instead, the system must balance five partially competing criteria:

Musical relevance: the selected motion should be associated with music that is similar in content, rhythm, and timbre to the live query.

Temporal compatibility: the motion should remain usable within the permitted speed range, and its salient poses should be compatible with the observed beat and onset density.

Responsiveness and stability: the system should react to a genuine musical change while avoiding oscillation caused by small ranking changes.

Motion continuity: switching should minimise discontinuities in joint position, joint velocity, support state, root pose, and root velocity.

Runtime feasibility: audio analysis, retrieval, transition planning, and motion playback must operate within the control-loop timing budget and respect joint and collision constraints.

These criteria cannot be reduced to a single nearest-neighbour query. They require a hierarchy in which retrieval, compatibility scoring, temporal stabilisation, transition planning, phase control, and output limiting are treated as related but distinct components.

The initial engineering design target is a 120 Hz foreground control loop, corresponding to an 8.33 ms frame period. Its empirical timing criteria are per-run control-work p99 below one frame period and a deadline-miss ratio below 1%, with final-output and readiness checks enabled. This target concerns pose updates, not the once-per-second retrieval schedule or the six-second audio context. Chapter 6 tests it in a capacity pilot. The subsequent formal evaluation uses a separately declared 60 Hz operating point, with a 16.67 ms frame period and the same 1% miss criterion. Success at 60 Hz establishes that operating point; it does not fulfil the 120 Hz design target.

1.4 Overview of the Proposed System

The proposed system contains an offline preparation path and an online control path. Offline, AIST++ motion files are paired with their synchronised music. Audio is divided into six-second windows and encoded using a gain-robust descriptor comprising a content embedding and explicit rhythm–timbre features. The current catalogue contains 411 motion profiles associated with 60 music identifiers and 348 audio segments. For each motion, salient poses are identified as prominent valleys in a whole-body velocity signal derived from SMPL joint rotation and root translation. The human sequence is then retargeted through GMR to the 29-degree-of-freedom Unitree G1 representation. Only canonical trajectories that pass metadata, continuity, joint-range, grounding, and collision-related checks are admitted to online selection.

Online, microphone input or a virtual microphone supplies both a rolling audio window and low-level beat events. A hierarchical matcher first compares the query with catalogue audio segments and aggregates segment evidence at the track level. It then ranks motions linked to the best tracks using tempo, key-pose-density, and activity compatibility. The selection policy requires stable evidence before a relevance-driven change and can rotate among near-optimal motions after a bounded number of held bars to preserve diversity. Candidate motions are loaded and prepared asynchronously.

At a switch boundary, the system evaluates possible entry phases near salient poses. The cost accounts for normalised joint-pose difference, joint-velocity difference, foot-contact mismatch, root-state difference, and motion salience. The selected target is aligned to the current world root and blended over a duration derived from joint speed and acceleration limits and quantised to a half-beat interval. During playback, accepted beats update the target phase rate and a bounded controller gradually consumes alignment error. A final output layer constrains joint dynamics and applies the configured runtime collision policy before the pose is written to MuJoCo.

This architecture differs from FACT, Bailando, EDGE, RoboPerform, and DiscoForcing in that it does not train an online motion generator. Instead, it performs causal retrieval and control over a finite library of robot-ready trajectories. The comparison is therefore not a claim that retrieval replaces generative modelling. Rather, the system explores an alternative operating point in which interpretability, predictable motion provenance, explicit constraints, and modest runtime cost are prioritised over the ability to synthesise novel choreography.

Chapter 2

Literature Review

2.1 Introduction

Music-driven humanoid motion combines music information retrieval, character animation, online control, and robotics. Although all four fields model temporal behaviour, they impose different requirements: audio analysis extracts musical structure, dance generation seeks plausible motion, real-time control requires causal and continuous responses, and humanoid deployment adds contact, kinematic, and feasibility constraints.

This chapter reviews these areas in relation to causal selection and playback of robot-ready dance from streaming audio. It distinguishes offline generation, which may use a complete music sequence; streaming generation, which restricts access to future audio; and retrieval-and-control, which adapts motions from a finite database. BeatWeaver belongs to the third category, drawing on work in music-to-dance generation, beat tracking, motion graphs, motion matching, and humanoid retargeting. The review focuses on the trade-offs among novelty, latency, interpretability, provenance, and embodiment safety that motivate its hybrid architecture.

2.2 Music Representation for Motion Control

2.2.1 Temporal and semantic information in music

A music representation for dance control must describe both when motion events should occur and what kind of movement may be appropriate. Rhythmic descriptors such as tempo, onset strength, beat confidence, and inter-beat interval provide temporal cues. Spectral shape, timbre, harmony, loudness, and learned style embeddings provide broader evidence about musical content and activity. These two groups are related but not interchangeable.

Classical music information retrieval systems construct descriptors from short-time signal analysis. A frame-level descriptor sequence may be written as

$$\mathbf{f}_{1:T} = \mathcal{F}(\mathbf{a}_{1:S}), \quad (2.1)$$

where $\mathbf{a}_{1:S}$ is an audio waveform and $\mathcal{F}$ may include a short-time Fourier transform, mel-frequency bands, spectral statistics, chroma, onset strength, or rhythm summaries. Libraries such

as Essentia collect spectral, temporal, tonal, and high-level descriptors in a common analysis framework [6]. Librosa provides a Python implementation of reusable audio and music-analysis primitives [22]. BeatWeaver uses librosa and NumPy for its DSP descriptors; the Essentia citation provides library context rather than implementation attribution. Such descriptors are computationally attractive because their meaning and cost are explicit. They can also be normalised independently, which is valuable when microphone gain, playback volume, or recording conditions change.

Onset detection supplies evidence of local change, whereas beat tracking seeks a recurring temporal organisation of those changes. Bello et al. distinguish energy, spectral and phase-based onset cues, explaining why a single detection function cannot capture every kind of musical attack [4]. This motivates using onset density and spectral flux alongside estimated tempo. Harmonic/percussive separation supplies another complementary cue: FitzGerald uses median filtering along time and frequency to distinguish sustained harmonic structure from transient percussion [9]. BeatWeaver applies the librosa separation primitive to a mel representation to derive a percussive ratio. That ratio is an engineered activity cue, not a validated measure of perceived dance energy.

However, hand-designed features expose an important design tension. Global summary statistics are cheap and stable, but they can discard local structure. Frame-level features retain that structure, but direct comparison becomes more expensive and sensitive to alignment. The appropriate representation therefore depends on the decision timescale. A short rolling window is useful for responsiveness, whereas a longer context is useful for estimating stable style or tempo. A real-time system commonly needs both. Local events for phase control, and a slower descriptor for motion selection.

2.2.2 Learned music embeddings

Learned embeddings replace or complement manually chosen descriptors by mapping audio into a feature space whose geometry is shaped by a training objective. Music representations can be supervised by genre and mood labels, by co-occurrence information, or by metadata-derived similarity. In particular, Alonso-Jiménez et al. use associations extracted from Discogs editorial metadata as contrastive supervision and show that large, weakly labelled music collections can produce useful general-purpose representations [2]. This line of work is relevant to the Discogs EffNet backend used by the current BeatWeaver catalogue.

A learned representation nevertheless introduces new sources of uncertainty. Its similarity metric reflects the pretraining data and objective rather than the requirements of robot motion. A track-level style embedding may rank two pieces of music as similar even when their beat rates would require an infeasible change in motion speed. Domain shift is also possible when a model trained on mastered recordings receives reverberant or clipped microphone audio. For this reason, semantic embedding similarity should not be treated as a complete cross-modal compatibility function. A hybrid score can retain the coverage of learned features while applying explicit tempo, rhythm, and activity terms whose contribution can be inspected and ablated.

Let $\mathbf{e}_q$ and $\mathbf{e}_i$ be standardised query and catalogue embeddings. A typical content similarity is their cosine similarity,

$$s_{\text{emb}}(q, i) = \frac{\mathbf{e}_q^\top \mathbf{e}_i}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_i\|_2}. \quad (2.2)$$

For motion control, this term is more naturally interpreted as evidence for a related musical neighbourhood than as direct evidence that a particular trajectory is executable. The distinction motivates hierarchical retrieval: first identify relevant music, then rank the motions associated with that music using motion-specific compatibility.

2.2.3 Beat and onset analysis under causal constraints

Beat tracking illustrates the difference between analysing music and controlling motion from music. Ellis formulates beat tracking as a compromise between local onset evidence and a transition preference for a regular inter-beat interval [8]. This formulation captures a general principle that a beat is not merely a spectral peak, but a temporally coherent hypothesis supported by recent events. For offline analysis, dynamic programming can optimise a sequence of beat times over a complete region. But a live controller cannot freely revise past beat decisions after the corresponding robot pose has already been emitted.

Grosche and Müller’s predominant local pulse (PLP) method estimates local periodicities from onset information and combines them into a pulse function [11]. It supplies the algorithmic basis for BeatWeaver’s rolling-window beat candidates through librosa’s PLP implementation. Computing PLP on acquired audio preserves the no-future-input constraint, but local analysis and asynchronous delivery still introduce delay. An estimated beat position and the time at which the controller receives it are consequently different quantities. This distinction motivates the delivered-event metric and explicit timing logs in Chapter 6.

Learned rhythm analysis offers a complementary direction. The madmom library includes neural and probabilistic methods for onset, beat, downbeat and metre analysis [5]. Its distinction between those tasks highlights a limitation of treating every fourth accepted pulse as a bar boundary: counting pulses does not itself estimate metre or downbeats. Madmom is a literature comparison and a possible future baseline; it is not the beat backend evaluated in this dissertation.

Therefore, causal beat handling requires explicit confidence and latency policies. Strong percussive subdivisions can produce twice the intended control rate, while weak passages can produce missing or unstable events. Accepting every detected event risks aligning motion key poses to hi-hat subdivisions rather than perceptually strong beats. Rejecting too many events causes stale tempo estimates and slow phase correction. Relative onset or loudness contrast over recent beats offers a causal way to prefer stronger events, but its quality depends on the length and diversity of the available history.

The important consequence for the present research is that audio retrieval and beat-phase control should use related but separate signals. Retrieval benefits from several seconds of context and should not change at every onset. Phase control requires low-latency event timing but should not reinterpret every event as a change of musical identity. This separation reduces the risk that a single noisy audio block simultaneously changes the selected dance and its playback phase.

2.3 Dance Data and Motion Representation

2.3.1 Paired music–dance datasets

Progress in music-conditioned dance generation has been strongly influenced by the scale and representation of available paired datasets. AIST++ contains 1,408 three-dimensional dance sequences across ten genres and is accompanied by aligned music and camera information [18]. Its size, genre labels, and synchronisation make it a common benchmark for music-conditioned generation and a practical source for the catalogue in this dissertation. AIST++ is especially relevant to retrieval-based work because multiple dance sequences can be associated with related musical material, permitting the catalogue to represent one-to-many music–motion relationships.

AIST++ reconstructs human motion from the AIST Dance Video Database [27]. The ten genre names and identifier codes used here are breaking (BR), house (HO), ballet jazz (JB), street jazz (JS), krump (KR), LA-style hip-hop (LH), locking (LO), middle hip-hop (MH), popping (PO), and waacking (WA). These are dataset dance categories, not an exhaustive taxonomy of music. The same codes label the genre retrieval results in Table 6.3.

Figure 2.1 shows the relationship between the two contributions that are often mentioned together: AIST++ provides paired music and reconstructed human motion, whereas FACT learns to generate future human poses from that data.

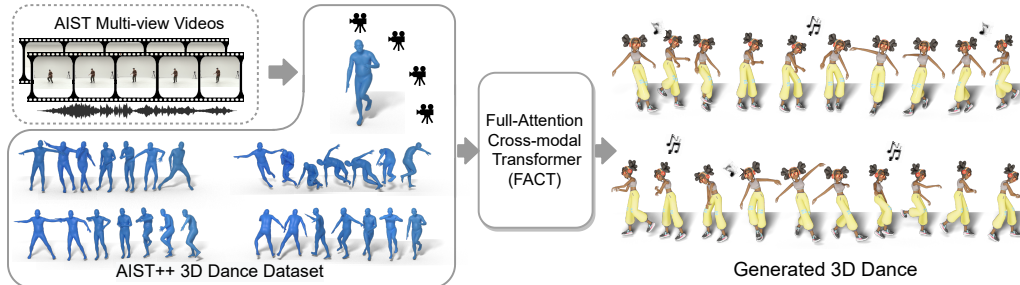


Figure 2.1: The AIST++ dataset and FACT generation setting, reproduced from Fig. 1 of Li et al. [18]

FineDance extends the emphasis from broad coverage to fine-grained, full-body choreography. It increases stylistic diversity and includes more detailed hand movement and genre coverage than earlier benchmarks [17]. The accompanying method also uses retrieval as contextual assistance for generation, illustrating that retrieval and generative modelling need not be mutually exclusive. Retrieval can supply long-range examples or style context, while a generator produces local motion.

Dataset scale does not remove ambiguity from evaluation. Music and dance have a many-to-many relationship, so a generated or retrieved movement can be appropriate without matching a held-out reference pose. Dataset labels may also reflect dance genre, music genre, performer, or recording session in different proportions. Models can consequently exploit correlations that do not transfer to live music. Evaluations based only on reconstruction distance are therefore insufficient. Rhythmic alignment, distributional quality, diversity, transition continuity, and user judgement provide complementary evidence.

Distributional evaluation also needs a stated representation and sampling protocol. Fréchet

Inception Distance (FID), introduced by Heusel et al., compares Gaussian feature distributions [12]. AIST++/FACT and Bailando adapt this evaluation family to kinetic and geometric motion features; the corresponding diversity (Div) measures describe distances among motion feature vectors [18, 24]. These scores capture distributional resemblance and spread rather than a unique correct choreography. Chong and Forsyth show that finite-sample FID estimates can have model-dependent bias [7]. Thus matching sample counts is necessary but does not make a small reconstructed retrieval sample a leaderboard-equivalent benchmark. Chapter 6 defines both metrics, their feature spaces and their sampling and normalisation conventions explicitly.

2.3.2 Human-body and robot-motion representations

Human dance datasets frequently represent motion through skeletal joint positions, joint rotations, or a parametric body model. SMPL defines a skinned human model with pose and shape parameters and is compatible with standard graphics pipelines [20]. It provides a useful canonical human representation because rotations preserve articulated structure and the model can generate consistent joint and surface geometry across performers.

For a target humanoid, however, an SMPL pose is not a command. A robot has a different kinematic tree, joint axes, degrees of freedom, proportions, and collision geometry. A representation suitable for visual generation may also omit velocities, contact states, actuator limits, or world-frame root requirements. Thus, the apparent output type “3D motion” conceals a major difference between human animation and robot execution.

The representation selected for a catalogue also affects matching and transition costs. Raw joint angles allow direct comparison in the target configuration space but require meaningful normalisation across joints. Cartesian joint positions make geometric similarity clearer but can hide joint limit differences and amplify root-alignment errors. Velocities reveal whether two poses are moving toward compatible future states. Foot contacts encode a discrete support condition that neither pose nor velocity alone captures. Consequently, a robot transition should normally consider a structured state rather than a single pose vector.

2.4 Music-Conditioned Dance Generation

2.4.1 Autoregressive and discrete motion models

FACT couples a full-attention cross-modal transformer with the AIST++ dataset to model long music and motion sequences [18]. Its transformer-based formulation demonstrates that temporal context and cross-modal attention can capture dependencies beyond a local beat. At the same time, autoregressive generation exposes familiar difficulties: errors can accumulate over long horizons, and a model conditioned on previously generated frames can become sensitive to its own imperfect history.

Bailando addresses choreographic structure through a learned discrete motion codebook and an actor-critic GPT framework [24]. The codebook turns continuous pose sequences into motion tokens, while the reward design encourages beat alignment. Discrete representations can make long-range modelling more manageable and can preserve recurring movement patterns. However,

token boundaries do not automatically correspond to dynamically safe robot transition points, and decoding a human pose sequence still leaves an embodiment conversion problem.

These methods establish two important ideas for the BeatWeaver. First, musical context beyond the instantaneous beat is necessary for coherent selection. Second, repeated motion patterns are not inherently undesirable, because dance contains motifs whose recurrence supports style and structure. A retrieval system can exploit this property directly, although its available motifs remain limited to the catalogue.

2.4.2 Diffusion-based dance synthesis

Diffusion models treat motion generation as iterative denoising conditioned on audio and other signals. Listen, Denoise, Action! applies this formulation to audio-driven motion and supports probabilistic sampling rather than a single deterministic output [1]. EDGE uses a transformer diffusion model for music-conditioned dance and demonstrates both high-quality generation and editing operations such as motion in-betweening [26]. Beat-It introduces explicit beat and key-pose conditions, showing the value of representing rhythmic constraints directly rather than expecting them to emerge only from a global audio embedding [14].

Figure 2.2 illustrates why these conditions are distinct. Music supplies dense contextual evidence, while beats and key poses identify sparse temporal requirements that should survive the generation process.

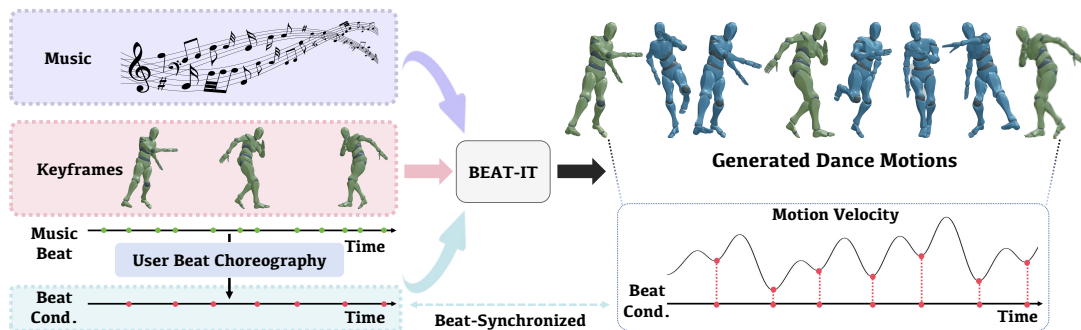


Figure 2.2: Beat-It’s music, key-pose, and beat conditioning, reproduced from Fig. 1 of Huang et al. [14]

The principal attraction of diffusion is its capacity to represent multiple valid motions for the same music. This is well matched to the multimodal nature of dance. Iterative sampling can also integrate diverse constraints. The cost is that denoising introduces computation and, in a conventional formulation, often assumes a fixed sequence whose future frames are revised together. A motion that looks coherent after full-sequence denoising does not by itself establish causal, bounded-latency behaviour.

For this reason, offline diffusion results and real-time control results should not be compared solely through motion quality metrics. They solve different information problems. If an offline model observes future musical events, it can anticipate an upcoming accent. A causal controller

can respond only after sufficient evidence has arrived. The fair comparison must include latency, look-ahead, revision policy, and how committed motion history is handled.

2.4.3 Retrieval within generative systems

FineDance incorporates a retrieval mechanism to improve genre consistency and long-term coherence before or alongside generation [17]. This hybrid design points to a broader interpretation of retrieval. A database need not merely return a final clip; it can provide contextual priors, examples, or candidates that are subsequently adapted. The quality of retrieval then depends on the correspondence between the query feature space and the desired motion property.

In the BeatWeaver, retrieval has a stricter role: the selected item is a prevalidated robot trajectory rather than a context example for a generator. This reduces representational freedom but strengthens motion provenance. Every candidate can be inspected offline, linked to its source audio and retargeting metadata, and rejected if it fails a preflight check. The comparison suggests a continuum rather than a binary distinction. At one end, pure generation offers high novelty with less explicit provenance. At the other, pure retrieval offers strong provenance with finite coverage. Future systems may use retrieval to propose safe motion primitives and generation to adapt them within controlled bounds.

2.5 Real-Time Audio-Driven Control

2.5.1 Causality, latency, and committed history

For an online system, causality can be expressed as

$$(i_t, \phi_t, \mathbf{q}_t^{\text{out}}) = \mathcal{G}(\mathcal{A}_{\leq t-\delta}, \mathcal{M}, \mathcal{H}_t), \quad (2.3)$$

where δ represents acquisition and processing delay, $\mathcal{M}$ is the available motion collection, and $\mathcal{H}_t$ is controller history, including previously emitted configurations $\mathbf{q}_{<t}^{\text{out}}$. The outputs are the selected motion index, phase and emitted configuration. Here $\mathcal{G}$ denotes online control, while $\mathcal{F}$ above denotes feature extraction. A strictly causal system has no access to $\mathcal{A}_{>t}$, and a bounded-latency system must also place a practical upper bound on δ . In addition, previously emitted poses $\mathbf{q}_{<t}^{\text{out}}$ are committed: an algorithm may update its internal belief about the music, but it cannot alter the robot’s physical past.

This formulation reveals three distinct latency sources. *Perceptual latency* comes from the audio context needed to estimate a reliable feature. *Computational latency* comes from feature extraction, inference, search, and planning. *Scheduling latency* comes from deliberately waiting for a musically or mechanically suitable switch boundary. Reducing only model inference time does not guarantee a responsive system if a six-second feature window is recomputed infrequently or if every change waits several bars.

2.5.2 Direct audio-to-humanoid policies

Do You Have Freestyle? introduces RoboPerform, an audio-conditioned humanoid locomotion framework that aligns audio and motion representations and distills a mixture-of-experts teacher

into a diffusion-based student policy [19]. A central contribution is retargeting-free control: the student produces humanoid actions from audio rather than first generating an explicit human sequence and passing it through a separate retargeter. This can reduce accumulated error and latency across pipeline stages. The work also demonstrates expressive behaviour on a Unitree G1 platform, making it more directly relevant to humanoid performance than animation-only dance models.

RoboPerform and the BeatWeaver address related goals through different assumptions. RoboPerform learns an action policy and can synthesise behaviour beyond a fixed set of stored trajectories. The BeatWeaver assumes that a library of actions has already been retargeted and checked. The former places more of the problem in representation learning and policy training. The latter places more of it in catalogue construction, similarity design, selection stability, and transition planning. The contrast is useful because it identifies two different ways of handling embodiment: learn it inside the policy, or validate it before retrieval.

2.5.3 Streaming diffusion forcing

DiscoForcing explicitly formulates audio-driven character motion as strict causal streaming with bounded latency [15]. It combines a causal music encoder with diffusion forcing, a hybrid temporal training schedule, and history-guided sampling. The model is designed to remain responsive to new audio while preventing long-term degradation from stale or imperfect generated history. Its demonstrations span animated characters and humanoid deployment, illustrating that streaming generation can be treated as a first-class problem rather than an offline model run on shorter windows.

The importance of DiscoForcing to this dissertation is methodological. It provides a stronger standard for the term “real time”: causal access, specified latency, continuous history, and online output should all be stated. The BeatWeaver does not use diffusion forcing, but it faces analogous stability questions. A generative model must decide how much noisy history to condition on. A retrieval system must decide how much ranking history is enough to justify a switch. Both systems balance responsiveness against temporal consistency, although they implement that balance at different levels.

There is also an important difference in failure behaviour. A streaming generator may gradually drift or produce locally implausible motion, whereas a retrieval system is bounded by the quality of its stored clips but can still fail through a poor match, phase error, or discontinuous transition. Therefore, the evaluation metrics should differ. Generative diversity and distributional quality are central for DiscoForcing. Catalogue recall, switching stability, entry-state cost, and deadline misses are central for the BeatWeaver.

2.6 Example-Based Motion Synthesis and Motion Matching

2.6.1 Motion graphs and database reuse

Long before modern audio-conditioned diffusion, data-driven animation investigated how captured motions could be rearranged without losing their realism. Motion graphs construct a directed graph whose edges contain captured motion or generated transitions and whose walks form new

sequences [16]. The critical operation is the discovery of states at which one motion can transition into another with limited visual discontinuity. This reframes motion synthesis as search over reusable examples.

Motion graphs are conceptually relevant to a finite dance library, but a dance matcher introduces additional structure. Candidate transitions are not chosen only to follow a user-specified spatial path; they are conditioned by music, beat boundaries, motion salience, and humanoid support state. Moreover, a transition graph built from every compatible frame pair may become large. The present system instead performs a local entry search when a motion switch is accepted. This avoids storing a dense graph while retaining the central insight that database clips should be joined at compatible states rather than arbitrary timestamps.

2.6.2 Motion matching and learned compression

Motion matching searches a motion database at runtime for a frame whose pose and future trajectory best fit the current character state and desired control. Learned Motion Matching compresses this behaviour into neural networks while retaining the responsive, data-driven character of database search [13]. The work highlights the main strengths of example-based control: captured detail can be preserved, behaviour is guided by an explicit query, and selection can respond continuously to an external target.

The BeatWeaver uses the term *Weaver* in a related but more hierarchical sense. Its primary query is music rather than a desired locomotion trajectory, and it initially ranks complete motion candidates associated with catalogue tracks. Only after a candidate is selected does it search entry states using the current robot pose and velocity. This division addresses a scaling and semantics problem. Joint-state proximity alone cannot establish that a dance matches the current music, while music similarity alone cannot establish that the transition is continuous.

A general hierarchical score can be expressed as

$$i^* = \arg \max_{i \in \mathcal{C}(q)} [w_m s_{\text{music}}(q, i) + w_r s_{\text{tempo}}(q, i) + w_k s_{\text{key}}(q, i) + w_a s_{\text{activity}}(q, i)], \quad (2.4)$$

where $\mathcal{C}(q)$ is a candidate set produced by track-level retrieval. This formulation makes the compatibility dimensions visible. It also supports ablations that would be harder to interpret if the entire cross-modal relation were represented by one end-to-end latent score.

2.7 Motion Transitions and Beat–Phase Synchronisation

2.7.1 State-compatible motion entry

Motion-graph and motion-matching research establishes pose similarity as a foundation for transition search [16, 13]. For humanoid dance, pose distance is necessary but incomplete. Two frames can have similar joint angles while their velocities point in opposite directions. Likewise, geometrically similar poses can place different feet in support, causing visible sliding or an abrupt change in the root trajectory.

A more complete entry cost for candidate phase ϕ is

$$C(\phi) = \lambda_q C_q(\phi) + \lambda_v C_v(\phi) + \lambda_c C_c(\phi) + \lambda_r C_r(\phi) + \lambda_s C_s(\phi), \quad (2.5)$$

where the terms measure joint pose, joint velocity, contact state, root state, and musical or key-pose salience. The weights expose a practical compromise. A large pose weight favours a visually smooth first frame. A large salience weight favours musically meaningful entry. Contact and root terms reduce foot and world-frame artefacts. The transition planner can restrict its search to salient phases, thereby reducing computation and avoiding entry in the middle of an expressive gesture.

2.7.2 Blending, root alignment, and output constraints

Once an entry state is chosen, blending can distribute the remaining mismatch over time. Database animation methods use automatically generated transitions to connect captured sequences [16]. For a floating-base humanoid, the target motion should first be aligned to the current world root. Otherwise a locally smooth joint blend can still cause a global translation or yaw jump. Joint interpolation should also respect angular topology and take the shortest path for wrapped angles.

The blend duration is not purely aesthetic. If the pose displacement for joint j is Δq_j , then a lower bound derived from a velocity limit $\dot{q}_j^{\max}$ is approximately

$$T_{\text{blend}} \geq \max_j \frac{|\Delta q_j|}{\dot{q}_j^{\max}}. \quad (2.6)$$

Acceleration limits can impose a longer bound. Quantising the final duration to a beat subdivision preserves a clearer musical structure, but can increase scheduling latency. A final limiter remains necessary because phase-speed adaptation, interpolation, and numerical sampling can combine to produce a larger step than any individual component predicts.

These mechanisms improve kinematic continuity but do not prove dynamic stability. A smooth joint-space path may still require infeasible torques or move the centre of mass outside a viable support region. This distinction is central when interpreting MuJoCo output later in the dissertation.

2.7.3 Beat-to-key-pose alignment

Music-conditioned generators often measure whether kinetic events coincide with musical beats, and Beat-It makes beats and key poses explicit conditioning variables [14]. A retrieval system faces the inverse problem: the motion already contains salient events, so playback timing must adapt those events to the incoming beat stream.

Instantaneously setting motion phase at every beat guarantees small phase error at the update instant but creates a discontinuity in motion time. A gradual controller instead modifies phase rate and consumes error over an interval. If ϕ_t is the current cyclic phase, $\hat{\phi}_t$ is the desired key-pose phase, and ω_t is playback rate, a simplified update is

$$\phi_{t+1} = \phi_t + \Delta t \text{ clip}\left(\omega_t + k_p \text{ wrap}(\hat{\phi}_t - \phi_t), \omega_{\min}, \omega_{\max}\right). \quad (2.7)$$

The bounded rate prevents an accurate beat estimate from demanding an unacceptable motion speed. The controller must also distinguish beat tracking error from structural incompatibility. If a motion has too many or too few salient poses for the observed rhythm, phase correction alone cannot solve the problem; the motion should instead receive a lower compatibility score during selection.

2.8 Humanoid Motion Retargeting and Validation

2.8.1 The embodiment gap

Gleicher’s retargeting formulation treats changes of character proportions as a constrained motion-editing problem, preserving features of the source performance while adapting it to a different body [10]. This provides a foundation for the embodiment problem: source pose similarity alone does not preserve a world-space contact when limb lengths change. For a robot, joint ranges and mechanical geometry add further constraints, which motivates separate fidelity and feasibility checks.

Retargeting maps a source performance to a target body while attempting to preserve its semantic and perceptual character. For humanoid robots, this task is constrained by differences in morphology and mechanical capability. SMPL provides a consistent human source representation [20], but a Unitree G1 trajectory requires a target-specific floating base and 29 actuated joint coordinates. Direct copying is impossible because joint meanings, axes, and ranges do not correspond one to one.

GMR treats motion retargeting as a consequential part of the humanoid learning pipeline [3]. Its evaluation shows that artefacts such as foot sliding, self-penetration, and physically implausible reference motion can reduce downstream tracking robustness. This observation supports two design decisions in the BeatWeaver. First, retargeting occurs offline so that its cost and failures do not enter the live audio path. Second, the catalogue stores retargeting provenance and admits only trajectories that satisfy explicit preflight checks.

Figure 2.3 gives concrete examples of retargeting artefacts. Kinematic validation can reject these malformed robot references, but passing such checks still does not establish that a controller can track a trajectory while balancing under contact and torque limits.

Retargeting quality has at least three dimensions. Fidelity measures how well the target preserves the source gesture. Kinematic validity covers joint ranges, continuity, grounding, and collision geometry. Dynamic trackability concerns whether a feedback controller and physical robot can execute the reference under contact and torque constraints. A trajectory can pass the first two dimensions and still fail the third. The experiments in this dissertation should therefore avoid using kinematic success as evidence of physical deployment readiness.

2.8.2 Simulation and the limits of kinematic preview

MuJoCo is designed for multi-joint dynamics and contact-rich model-based control [25]. The capabilities of the simulator, however, do not determine how a particular application uses it. Writing a target pose directly to the simulator state and invoking forward kinematics visualises the robot configuration and allows geometric checks, but it bypasses actuator dynamics, balance control, and disturbance response.

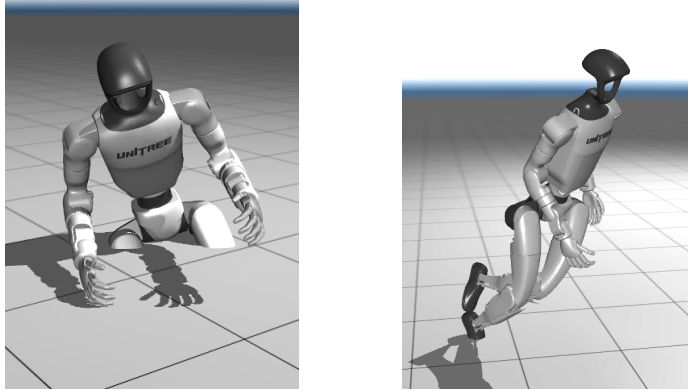


Figure 2.3: Ground penetration and self-intersection in retargeted humanoid references, reproduced from Fig. 2 of Araujo et al. [3]

DeepMimic demonstrates a different use of motion examples: reinforcement learning trains a controller to imitate reference skills through physical simulation [23]. The example trajectory guides behaviour, while the learned policy must produce actions under simulated dynamics. This distinguishes a reference library from an executable controller. BeatWeaver constructs and times robot references but does not train such a dynamics-based tracking policy, so its kinematic evaluation cannot establish the balance or disturbance-recovery capabilities studied in physics-based imitation.

This distinction prevents an otherwise common category error. A motion displayed in a physics engine is not necessarily a physics-controlled motion. Kinematic preview remains valuable. It can reveal joint-order mistakes, orientation errors, ground penetration, discontinuities, and self-collision before an online candidate is accepted. It can be reported as preflight and visualisation evidence, not as a sim-to-real result.

2.9 Critical Synthesis and Research Gap

Figure 2.4 provides a recent example of an end-to-end streaming architecture. It makes the distinction between causal audio feature extraction, streaming motion generation, online retargeting, and the final avatar or humanoid control stack visible in one pipeline.

The literature addresses important parts of music-driven motion without yet providing the integrated, lightweight solution required here. Offline models such as FACT, Bailando, EDGE, FineDance, and Beat-It demonstrate expressive, diverse, and structurally coherent music-to-human-motion generation [18, 24, 26, 17, 14], while DiscoForcing extends generation to causal streaming with bounded latency [15] and RoboPerform integrates embodiment through direct audio-conditioned humanoid control [19]. In parallel, motion graphs and motion matching establish principles for responsive reuse and state-compatible transitions [16, 13], and GMR shows that retargeted reference quality affects downstream humanoid tracking [3]. However, these strands do not jointly connect streaming music similarity, tempo and activity compatibility, stable selection,

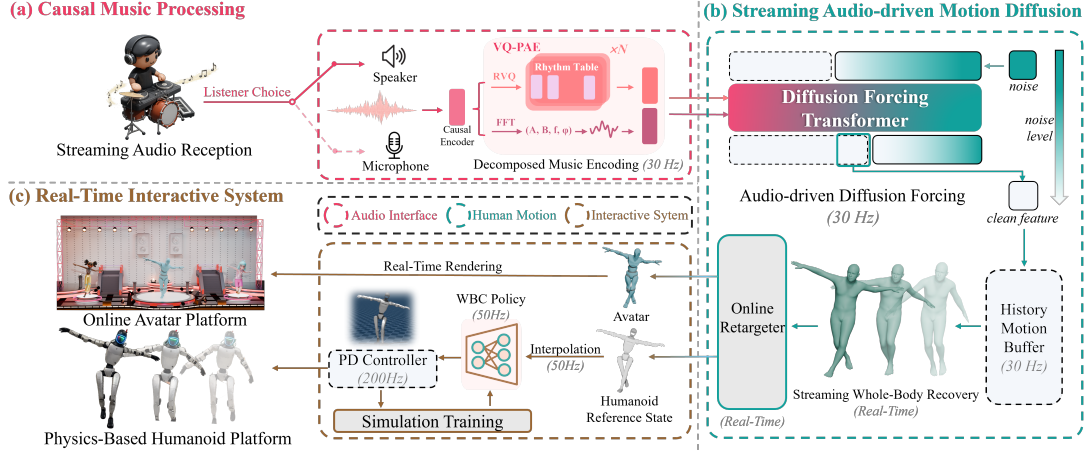


Figure 2.4: The DiscoForcing system pipeline, reproduced from Fig. 2 of Ji et al. [15]

beat-aware scheduling, phase synchronisation, continuous robot-state transitions, and validated motion provenance. This leaves a research gap for a system that treats these requirements as cooperating components rather than as a single similarity or generation problem. BeatWeaver addresses that gap by combining rolling audio analysis, hierarchical retrieval, interpretable compatibility scores, stateful switch acceptance, beat-boundary scheduling, key-pose phase control, state-aware entry search, root-aligned blending, and offline G1 validation. It investigates whether a finite prevalidated motion library can provide a causal, inspectable, and computationally modest operating point for real-time humanoid dance, while accepting less novelty than unconstrained generation.

Chapter 3

Overall System Design

3.1 Chapter Overview

This chapter presents the system-level design of the BeatWeaver. The system receives a causal audio stream and produces a continuous sequence of poses for a 29-degree-of-freedom Unitree G1 model. It does not synthesise a new dance from scratch. Instead, it retrieves music-related motions from a prevalidated catalogue, decides when a change is justified, adapts motion timing to musical events, and connects successive trajectories without an abrupt change in pose or root motion.

The central architectural decision is to separate an offline preparation path from an on-line decision-and-playback path. Computationally expensive or non-causal operations, including human-to-robot retargeting, whole-clip motion analysis, catalogue normalisation, and trajectory preflight, are performed offline. The online path is limited to rolling-window audio analysis, catalogue search, stateful selection, candidate preparation, beat-phase control, transition planning, and constrained pose output. A motion can enter the live candidate set only after its source, representation, and target-robot validity have been established.

The BeatWeaver is therefore a hybrid system rather than an end-to-end neural network. A pre-trained music model supplies learned audio representations, explicit DSP features describe rhythm and activity, deterministic scoring ranks catalogue items, and state machines govern selection and switching. Retargeted G1 trajectories supply the motion content, while a constrained controller governs timing and continuity. Each component has a narrow responsibility and exposes diagnostic values that can be tested independently. Chapter 4 specifies the offline data construction in detail, and Chapter 5 defines the online algorithms and their parameters.

3.2 End-to-End Architecture

Figure 3.1 summarises the architecture. The offline path supplies a frozen catalogue. The online path begins with a rolling audio buffer, proceeds through retrieval and selection, and ends with a robot pose after phase control, transition blending, and output checks.

The main information flow is

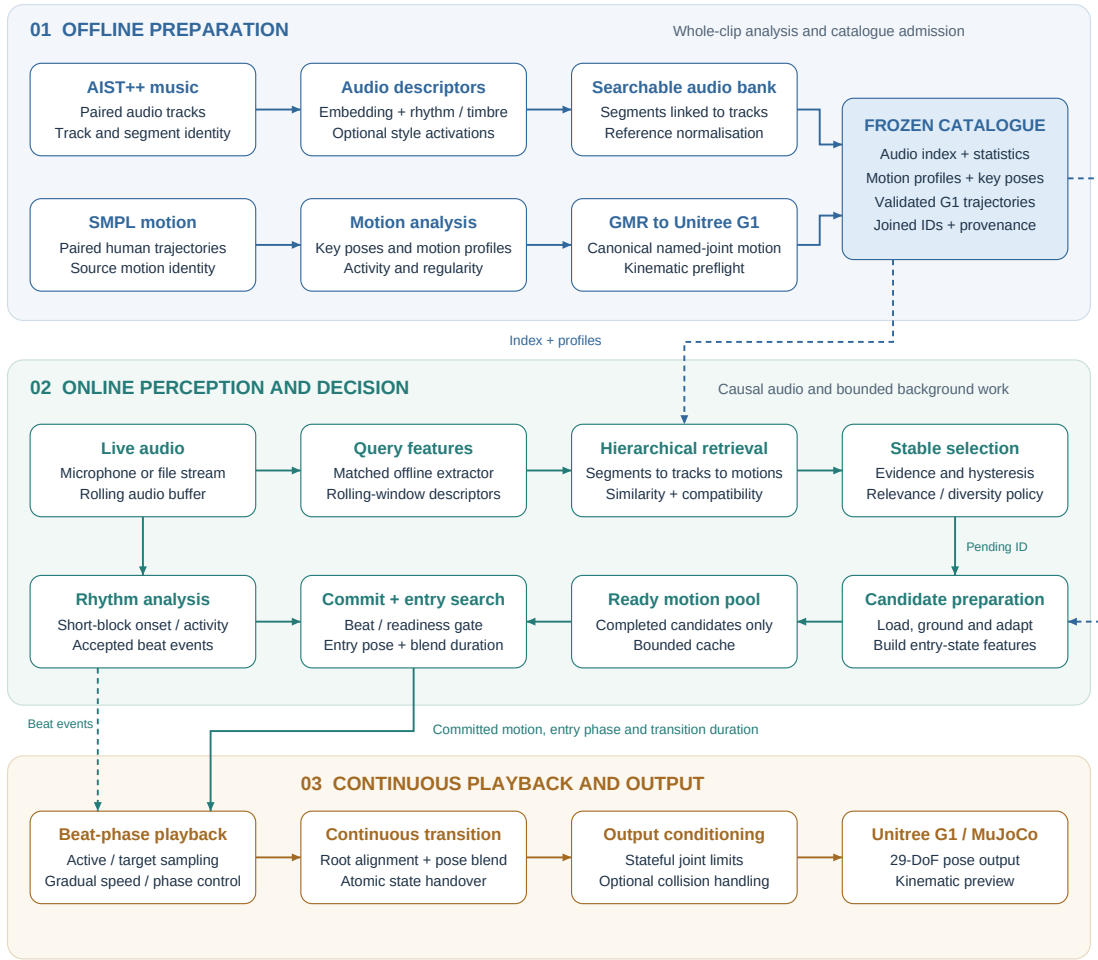


Figure 3.1: End-to-end architecture of the BeatWeaver

$$\begin{aligned}
\mathcal{A}_{\leq t} &\longrightarrow \mathbf{x}_t \longrightarrow \mathcal{T}_t \longrightarrow \mathcal{C}_t \longrightarrow d_t \\
&\longrightarrow (i_t, \phi_t, T_t) \longrightarrow \mathbf{q}_t^{\text{target}} \longrightarrow \mathbf{q}_t^{\text{out}},
\end{aligned} \tag{3.1}$$

where $\mathbf{x}_t$ is the descriptor of the recent audio, $\mathcal{T}_t$ is the ranked track set, $\mathcal{C}_t$ is the ranked motion set, and d_t is the stateful selection decision. A committed switch specifies a target motion i_t , an entry phase ϕ_t , and a transition duration T_t . $\mathbf{q}_t^{\text{target}}$ is the music-timed and, where necessary, blended target pose. $\mathbf{q}_t^{\text{out}}$ is the pose after final output conditioning.

The notation makes two boundaries explicit. A ranked motion is not yet a switch decision, and a switch decision is not yet a safe output pose. These boundaries prevent ranking jitter from changing the motion immediately, prevent an unloaded candidate from stalling the switch boundary, and prevent a relevant candidate from bypassing continuity and output checks.

The online architecture uses bounded background workers. Audio feature extraction and retrieval run outside the foreground playback path. Candidate motions are loaded, grounded, and prepared before the next eligible commitment point. The foreground path consumes only completed results and never waits for retargeting, feature extraction, or file loading. Coalescing retrieval work and bounding candidate caches prevents stale evidence or speculative preparation from accumulating without limit.

3.3 Subsystem Boundaries and Contracts

3.3.1 Offline preparation contract

The offline subsystem converts paired AIST++ music and SMPL motion [18, 20] into three linked catalogue units: audio segments, music tracks, and motion profiles. Segments provide local evidence for a live query, tracks aggregate related segments without rewarding duplicated or long recordings, and motion profiles distinguish choreographies associated with the same music.

Catalogue and query audio must use the same extractor identity, model revision, sample rate, window policy, and feature dimensions. Learned embedding and rhythm–timbre vectors are compared using statistics stored from the offline reference set. Style activations retain their label-coordinate meaning and are unit-normalised rather than z-standardised. This is a versioned interface, not an informal cache whose meaning depends on whichever code happens to load it.

On the motion side, the offline path detects a small set of salient entry hypotheses, computes activity and regularity summaries, and retargets the human sequence into a canonical named-joint G1 trajectory. Provenance and kinematic preflight determine admission. Only a joined record whose audio, source motion, profile, retargeted artefact, and preflight result are all present is exposed to the online matcher. The feature definitions, retargeting procedure, validation thresholds, and current catalogue statistics are given once in Chapter 4.

3.3.2 Online perception and selection contract

The online subsystem processes the same causal stream at two timescales. A longer rolling window supplies relatively stable evidence for music and motion retrieval, while short audio blocks supply

activity, onset, and beat events for timing.

Retrieval is hierarchical. Segment comparison first supplies local musical evidence, which is pooled at track level and may be restricted by style and abstention rules. A second stage evaluates whether linked motions are compatible with the observed tempo, event density, and activity. The component scores are retained because music similarity and motion feasibility have different failure modes. Their equations, weights, thresholds, and missing-evidence rules are defined in Chapter 5 rather than repeated here.

Selection adds temporal state to these rankings. Sustained evidence can create a pending relevance selection, whereas a separate bounded-hold policy may request diversity among similarly relevant motions. Both policies are subject to the same readiness and commitment rules. Rejected or weak audio clears new relevance accumulation and holds the active motion instead of forcing an arbitrary catalogue item.

3.3.3 Prepared switching and execution contract

A pending identifier expresses intent but does not authorise immediate playback. Preparation loads and validates the canonical trajectory, grounds it in the playback model, adapts its named joints to the active model, and builds the state features required for entry evaluation. Accepted beats define the musical commitment points shared by selection and phase control. If the preferred item is not prepared, the system may use only another ranked and prepared candidate. Otherwise it continues or safely completes the current trajectory.

The logical progression is

$$\text{current} \rightarrow \text{evidence} \rightarrow \text{pending} \rightarrow \text{prepared} \rightarrow \text{transitioning} \rightarrow \text{current}. \quad (3.2)$$

At a valid commitment point, entry search compares the current state with a small neighbourhood around salient target poses. Pose, velocity, support, root-state, and musical-salience disagreement determine entry quality. The transition duration then reflects the required joint displacement, configured dynamics limits, remaining target duration, and musical timing. These calculations are online concerns and are specified in Chapter 5.

During playback, accepted beats modify motion speed and consume phase error gradually. They do not teleport the sampler to a different pose. Target root motion is aligned to the current world trajectory before blending. One-shot clips do not cross an implicit last-to-first seam, and completion transfers the sampler, phase controller, root references, and motion identifier atomically. The resulting pose passes through stateful joint limits. Runtime collision handling, when enabled, is applied to the pose that would otherwise be emitted and must update the limiter state if it changes that pose. Offline preflight does not by itself imply that this optional online collision policy is active.

3.4 State Ownership and Failure Isolation

The system assigns each form of state to one responsible component. The retriever owns the latest completed ranking and its rejection status. The selector owns evidence streaks, pending

reason, held-bar state, and recency history. The preparation pool owns candidate readiness and load failures. Each motion controller owns phase, accepted-beat history, amplitude state, and rate limits. The transition controller owns source–target blending and transfers the new active state only at completion. Finally, the output layer owns the previous emitted joint state used by its dynamics limiter.

Figure 3.2 illustrates a representative asynchronous switch. The foreground columns distinguish logical state owners within the same playback loop, rather than independent execution threads. Retrieval and preparation return completed results while active-motion sampling and output conditioning continue. Candidate readiness and an eligible musical boundary permit commitment; the active playback state changes at blend completion. Background entry scoring uses a generation check to discard plans invalidated by newer retrieval evidence.

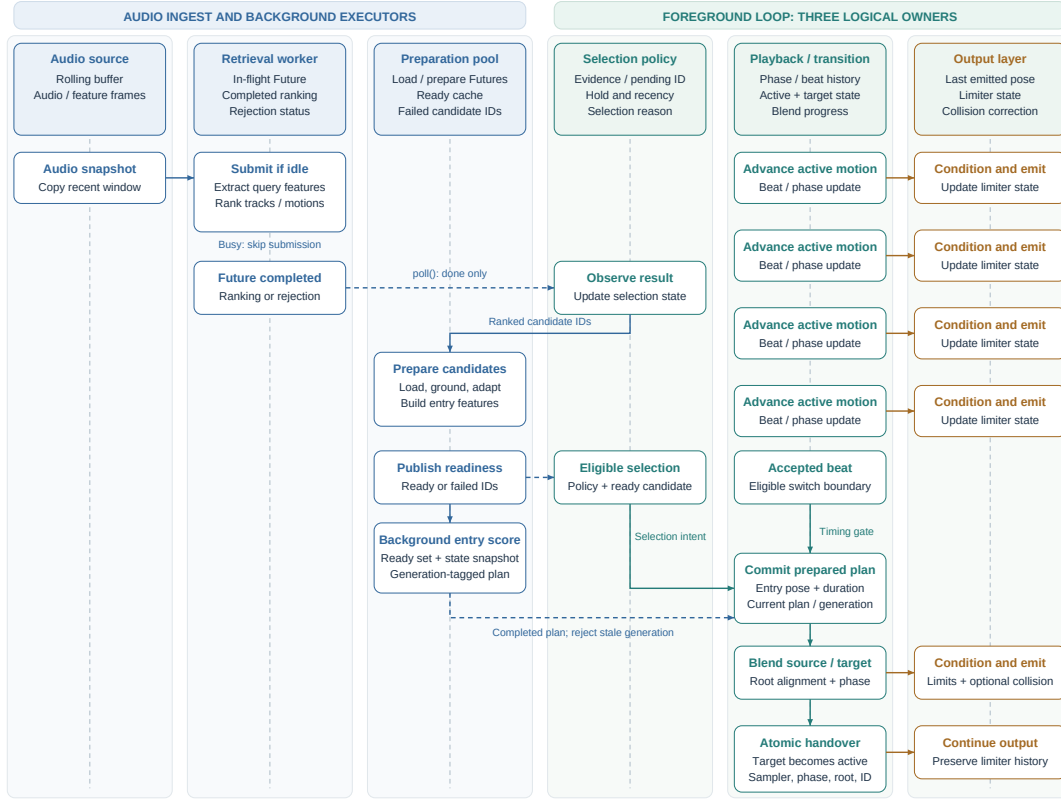


Figure 3.2: State ownership and asynchronous execution during a representative prepared switch

This separation prevents a partial failure from corrupting the active motion. A busy retrieval worker skips a submission opportunity rather than building a backlog. A loading or grounding failure removes the affected candidate from the ready set. A rejected query leaves the current motion active. If a one-shot clip ends before another evidence-backed candidate is prepared, its terminal pose is held instead of wrapping through an unauthorised seam or blocking on file I/O.

Diagnostics follow the same ownership boundaries. Retrieval exposes rankings and rejection

reasons. Selection exposes evidence, pending reason, and readiness. Timing exposes accepted beats and remaining phase error. Transition planning exposes entry quality and blend progress, and output conditioning exposes limiter and collision interventions. Chapter 5 defines the concrete trace fields used by later experiments.

The present output is a kinematic trajectory preview and geometric validation path in MuJoCo [25], not a torque-controlled balance simulation. Canonical retargeting, kinematic preflight, and optional online collision handling improve representation and geometric validity, but they do not prove dynamic trackability or hardware safety. Those claims require a separate controller and experimental evaluation.

Chapter 4

Offline Data Preparation and Catalogue Construction

4.1 Chapter Overview

The BeatWeaver can make a reliable decision only if its candidate motions and reference music have already been converted into consistent, validated representations. This chapter describes that offline preparation process. The input is the paired music and motion data provided by AIST++, in which a dance is represented by an SMPL pose sequence and is associated with a music recording [18, 20]. The outputs are two connected resources: a searchable audio catalogue and a set of canonical Unitree G1 trajectories. A catalogue entry is admitted only when both resources are available and the robot trajectory passes a kinematic preflight in MuJoCo.

The offline pipeline has four principal stages. First, source files and model assets are identified by stable names and cryptographic hashes. Second, audio is divided into overlapping analysis windows and encoded by learned and hand-designed descriptors. Third, SMPL motion is analysed to locate low-velocity key poses and is retargeted through General Motion Retargeting (GMR) to the 29-degree-of-freedom G1 model. Finally, the retargeted trajectory is validated, and the resulting descriptors, motion profiles, provenance records, and dense arrays are written as a versioned catalogue.

4.2 Source Data and Reproducibility Contracts

4.2.1 AIST++ pairing and source identity

AIST++ provides diverse 3D dance sequences with music and SMPL parameters, making it suitable for building a retrieval catalogue rather than a single genre-specific demonstration [18]. The preparation program reads the official split lists and associates each motion identifier with its SMPL motion file and music identifier. A row is not silently admitted when one side of this pair is missing. Instead, incomplete or invalid rows are recorded separately so that catalogue size cannot be mistaken for successful processing coverage.

The dataset contains repeated uses and variants of the same underlying music. Treating every file as an independent song would bias nearest-neighbour search towards duplicated recordings. The audio preparation stage therefore groups variants by music identifier, situation label, and a fingerprint computed from the first six seconds of decoded audio. Within an identical group, the longest recording is retained as the representative variant. This preserves meaningful recording differences while removing byte-level or content-level duplication that would otherwise inflate a track’s influence.

Every generated motion artefact records the source motion identifier and the SHA-256 digest of the source file. The prepared SMPL model, the GMR software revision, the target model, and important solver settings are likewise part of the generation contract. Consequently, equality of filenames is not treated as evidence that two artefacts have the same meaning. A cached trajectory is reused only if its provenance and schema agree with the current build request.

4.2.2 Preparation of the SMPL body model

SMPL represents the human body with a skinned surface, a 24-joint kinematic structure, pose parameters, and shape parameters [20]. The model files required to interpret these parameters are licensed assets and are not embedded in the project. A dedicated preparation step converts legacy array objects into ordinary NumPy arrays and verifies the structural fields used by the forward-kinematics code. In particular, the implementation checks a $6,890 \times 3$ template mesh, a $6,890 \times 24$ skinning-weight matrix, a $24 \times 6,890$ joint regressor, a 2×24 kinematic tree, and compatible shape directions. All arrays must be finite.

Structural validation alone cannot detect every damaged or incompatible model file. The prepared neutral model is therefore instantiated once in its rest pose. The expected 6,890 vertices and at least 24 finite joints must be produced before the model is accepted. Files are written atomically only after these checks succeed. This model preparation is intentionally separated from the dataset build: licensing and one-time format conversion are deployment concerns, whereas motion retargeting must be repeatable without manually editing model data.

4.3 Audio Catalogue Construction

4.3.1 Windowing and robust signal preparation

All audio is decoded to mono at 16 kHz. A track is divided into six-second windows with a two-second hop. A short track is zero-padded, and the last window is included even when the regular hop grid does not end exactly at the track boundary. This policy provides full temporal coverage while keeping the online query duration identical to the offline reference duration.

Before feature extraction, the median sample value is removed to suppress a DC offset. The signal is then scaled by its 95th percentile absolute amplitude and clipped to $[-1, 1]$. If $x[n]$ denotes the decoded signal and $q_{0.95}$ denotes the 95th percentile of $|x[n] - \text{median}(x)|$, the normalised waveform is

$$\tilde{x}[n] = \text{clip}\left(\frac{0.8(x[n] - \text{median}(x))}{\max(q_{0.95}, \epsilon)}, -1, 1\right). \quad (4.1)$$

Percentile scaling is less sensitive than peak normalisation to one impulsive sample. The factor 0.8 leaves headroom before clipping. Signal RMS and dynamic range are measured from the unnormalised signal, whereas learned and DSP descriptors use the robustly normalised waveform. This distinction prevents recording gain from dominating similarity while retaining loudness information as optional diagnostic metadata.

The six-second, two-second policy creates overlapping reference segments. This is deliberate. A music file may contain an introduction, a breakdown, and a dense rhythmic passage that should not share one global descriptor. Segment retrieval can first locate the locally relevant passage, after which evidence may be aggregated at track level. The resulting catalogue counts are reported with the final audio-motion join in Section 4.7.2, where their admission status can be interpreted correctly.

4.3.2 Learned music representation

Each window is passed through the official Discogs EffNet ONNX model. Models trained with Discogs editorial metadata provide embeddings organised around musically meaningful style information rather than generic acoustic events [2]. The implementation obtains two outputs from the same network:

- (1) a 1,280-dimensional embedding used for broad content and style similarity.
- (2) a 400-dimensional vector of style-label activations used as an interpretable tag representation.

Using one fixed model for both outputs avoids an undocumented combination of incompatible learned spaces. The catalogue metadata records the backend name, model filename, and model digest. The online extractor must satisfy the same contract. A change of model weights is consequently a catalogue migration. Existing reference vectors must be rebuilt before they can be compared with new queries.

4.3.3 Rhythm, timbre, and activity descriptors

Learned embeddings are strong descriptors of musical content, but tempo and rhythmic intensity need explicit treatment because they directly affect motion speed and activity. The project therefore computes complementary DSP features using short-time analysis with a 256-sample hop. The feature set includes tempo, beat strength, onset density, off-beat ratio, tempo stability, spectral flux, percussive ratio, three spectral-band energy ratios, mel-frequency cepstral coefficients (MFCCs), and spectral contrast. These primitives are implemented with librosa and NumPy, including librosa’s dynamic-programming beat tracker [22, 8]. The percussive ratio uses librosa’s median-filter harmonic/percussive separation on the mel representation [9]. This is a task-specific descriptor aggregation; Essentia is not the DSP feature-extraction library used here.

The scalar activity estimate is

$$a = \text{clip}_{[0,1]} \left(0.30b_s + 0.30 \min \left(\frac{d_o}{4}, 1 \right) + 0.20f_s + 0.20r_p \right), \quad (4.2)$$

where b_s is beat strength, d_o is onset density in events per second, f_s is normalised spectral flux, and r_p is the percussive energy ratio. The beat-quality estimate is

$$q_b = \text{clip}_{[0,1]} \left(0.40b_s + 0.35s_t + 0.25 \min \left(\frac{d_o}{4}, 1 \right) \right), \quad (4.3)$$

where s_t measures tempo stability. These quantities have separate meanings. Activity estimates how much motion energy the audio can support. Beat quality estimates whether tempo-dependent decisions should be trusted. For example, a noisy, onset-rich passage may have high activity but an unstable beat.

For compact comparison, the main handcrafted features are assembled into a 24-dimensional vector:

$$\mathbf{r} = [\log_2(B/120), b_s, d_o/4, r_o, s_t, f_s, r_p, \mathbf{e}_{\text{band}}^{(3)}, \boldsymbol{\mu}_{\text{MFCC}}^{(8)}, \boldsymbol{\mu}_{\text{contrast}}^{(6)}], \quad (4.4)$$

where B is tempo in beats per minute, r_o is the off-beat ratio, and the superscripts indicate the number of retained components. The logarithmic tempo term expresses octave-related tempo errors symmetrically. Estimates of 60 and 240 BPM are equally distant from 120 BPM in opposite directions. Per-track reference tempo is the median of its valid segment estimates, which is more robust than trusting a single window.

4.3.4 Dense array storage and normalisation statistics

The catalogue deliberately separates human-readable metadata from dense numeric data. JSON stores segment identities, time ranges, scalar descriptors, motion links, build configuration, and provenance. A compressed NPZ file stores the matrices required for vectorised search. In the current build, these arrays have dimensions

$$\mathbf{E} \in \mathbb{R}^{348 \times 1280}, \quad \mathbf{R} \in \mathbb{R}^{348 \times 24}, \quad \mathbf{T} \in \mathbb{R}^{348 \times 400}. \quad (4.5)$$

The file also stores a mean and standard deviation for every embedding and rhythm–timbre dimension. These statistics are computed only from the offline reference set and are reused online. They must not be recomputed from a live window, because doing so would make a query’s representation depend on itself and would destroy comparability across time. Zero or near-zero standard deviations are guarded before standardisation. Style activations retain their label-coordinate meaning and are unit-normalised online without z-standardisation. The JSON and NPZ paths are tied together by catalogue metadata so that a stale array file cannot be interpreted as belonging to a newly generated index.

4.4 Motion Key-Pose and Activity Analysis

4.4.1 Whole-body velocity signal

Key poses are useful entry candidates because a low-velocity posture generally allows a smaller transition error than an arbitrary high-speed frame. The implementation therefore derives a one-dimensional whole-body velocity signal from each source motion before retargeting. When 3D

joint positions are available, this signal is the root-mean-square linear speed across joints. The standard AIST++ representation used by the project contains SMPL rotations, so the default signal is instead based on geodesic angular velocity.

Let $R_{t,j} \in SO(3)$ be the world rotation of joint j at frame t , and let Δt be the frame interval. For an interior frame, the RMS angular speed is

$$v_{\text{ang}}(t) = \sqrt{\frac{1}{J} \sum_{j=1}^J \left\| \frac{\text{Log}(R_{t-1,j}^T R_{t+1,j})}{2\Delta t} \right\|_2^2}, \quad (4.6)$$

where $J = 24$ and $\text{Log} : SO(3) \rightarrow \mathbb{R}^3$ is the rotation logarithm. One-sided differences are used at the endpoints. An equivalent angular radius $r = 0.35$ m maps the result to an approximate linear-speed scale. If root translation is present, it is first divided by the SMPL scaling factor and combined as

$$v(t) = \sqrt{(rv_{\text{ang}}(t))^2 + (w_r v_{\text{root}}(t))^2}, \quad w_r = 1. \quad (4.7)$$

This signal is invariant to axis-angle wrapping because it operates on relative rotations rather than subtracting pose parameters directly. It also prevents a dance with stationary feet but rapid upper-body motion from being misclassified as a static sequence.

4.4.2 Valley detection and key-pose confidence

The raw velocity signal is smoothed by a Gaussian kernel with a standard deviation of 0.08 seconds. It is then robustly mapped to $[0, 1]$ using its 5th and 95th percentiles. Key-pose candidates are peaks of the negated normalised signal, and hence valleys in body velocity. The detector enforces a minimum temporal spacing of 0.28 seconds, a minimum prominence of 0.08, and a 0.10-second exclusion region at both sequence boundaries. These conditions remove shallow fluctuations and avoid selecting a boundary merely because the central-difference estimate is less reliable there.

For candidate k , confidence combines its peak prominence p_k with its normalised valley depth $d_k = 1 - \hat{v}_k$:

$$s_k = \text{clip}_{[0,1]}(0.65p_k + 0.35d_k). \quad (4.8)$$

Each retained key pose stores its frame index, time, normalised clip phase, score, prominence, and velocity. If a maximum count is requested, candidates are first selected by confidence and are then returned in temporal order. This avoids the common failure in which uniform temporal subsampling discards the only strong pause in a sequence.

The detector does not claim that every velocity minimum is a musically meaningful choreographic boundary. Instead, it constructs a set of mechanically favourable entry hypotheses. During online preparation, these hypotheses are ranked again using the current robot pose, velocity, contact state, and desired phase. Separating offline proposal generation from online entry evaluation keeps the live search small without fixing a transition choice prematurely.

4.4.3 Motion profile and catalogue groups

In addition to individual key poses, the analyser records summary statistics that are inexpensive to compare online. They include median and 90th-percentile velocity, clip duration, key-pose count, unweighted key-pose density, confidence-weighted density, and interval regularity. If the intervals between successive key poses are $\Delta\tau_i$, irregularity is measured by their coefficient of variation,

$$c_v = \frac{\text{std}(\Delta\tau_i)}{\text{mean}(\Delta\tau_i) + \epsilon}. \quad (4.9)$$

Low c_v indicates a more regular sequence of pauses, while high c_v indicates irregular phrasing. Weighted density distinguishes a clip with many weak numerical valleys from one with several pronounced key poses.

For catalogue organisation, motions are grouped by genre and coarse activity attributes. Median velocity and weighted key-pose density are divided at the dataset tertiles. Regularity is divided at its median. The present catalogue’s velocity cut points are approximately 1.10 and 1.51 ms^{-1} on the equivalent-speed scale, while density cut points are approximately 0.59 and 0.85 s^{-1} . These thresholds are derived from the catalogue. Their purpose is to support stratified diversity choices and interpretable diagnostics without requiring a learned motion-style classifier.

4.5 SMPL-to-G1 Retargeting

Figure 4.1 compares the source human mesh with the stored G1 trajectory at identical frame indices. The examples are selected from source-motion wrist separation and height, rather than from target-fit quality. They illustrate preservation of broad gestures alongside changes in body proportions and joint configuration; they are qualitative examples, not evidence of dynamic tracking or contact stability.

4.5.1 Human forward kinematics and coordinate conversion

The AIST++ pose at each frame contains local axis-angle rotations for the 24 SMPL joints and, when available, root translation. Retargeting individual local angles directly would ignore the different kinematic trees and proportions of the human and robot. The implementation therefore composes rotations through the complete SMPL hierarchy to obtain world-space joint positions and orientations. Root translation is divided by the sequence’s SMPL scale before being applied.

AIST++/SMPL motion is expressed in a Y-up convention, whereas the target MuJoCo model is Z-up. Positions and orientations are transformed by

$$C_{Y \rightarrow Z} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}. \quad (4.10)$$

World rotations are converted to scalar-first quaternions $\mathbf{Q} = [Q_w, Q_x, Q_y, Q_z]$. Because $\mathbf{Q}$ and $-\mathbf{Q}$ encode the same rotation, quaternion signs are made temporally continuous. This apparently small step is essential: an untreated sign flip appears as a large angular change to interpolation and velocity calculations even though the physical orientation is unchanged.

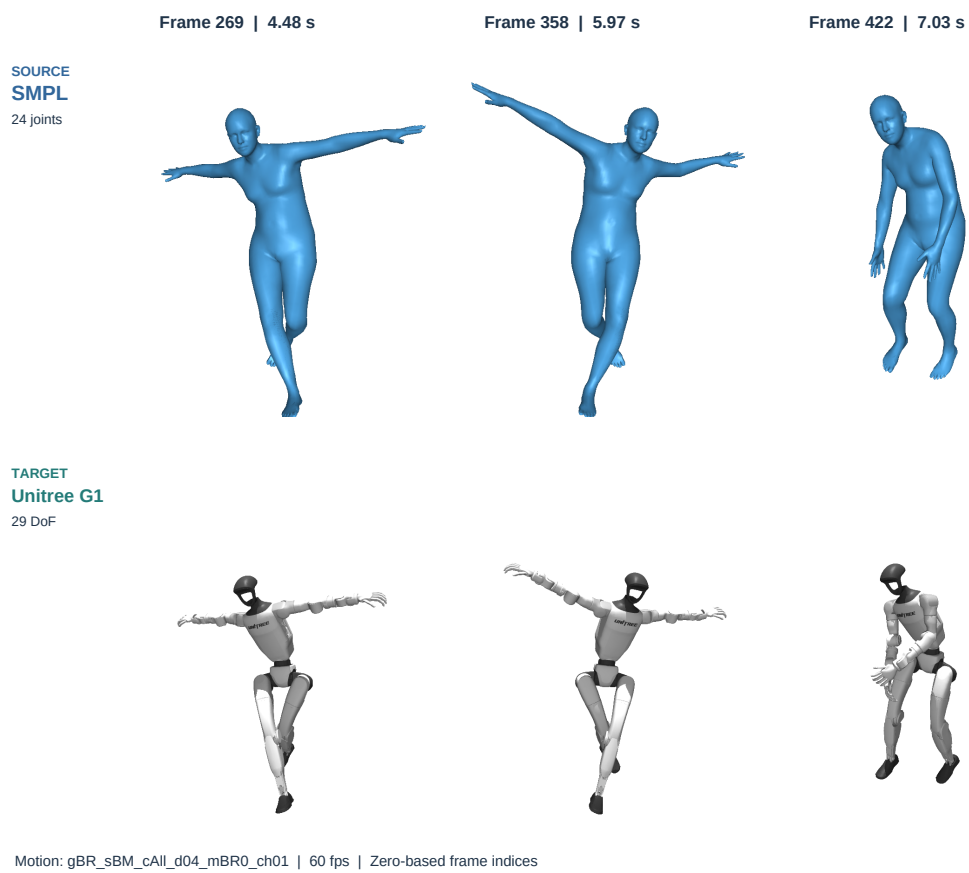


Figure 4.1: Same-frame SMPL-to-G1 retargeting comparison for AIST++ sequence gBR_sBM_cAll_d04_mBR0_ch01.

Fourteen human transforms are supplied as GMR targets: pelvis, upper spine, left and right hips, knees, feet, shoulders, elbows, and wrists. This subset captures root placement, leg contacts, torso orientation, and expressive arm motion while leaving GMR to resolve the different source and target skeletons. The method follows the general principle that retargeting quality materially affects the feasibility of subsequent humanoid tracking [3].

4.5.2 GMR inverse-kinematics solve

The target is the free-root Unitree G1 model with 29 actuated joints. For every source frame, GMR solves a constrained inverse-kinematics problem that balances the selected human transforms against the robot kinematic structure and joint limits. The implementation uses the DAQP-based solver with velocity limiting. A symmetric two-step arm seed is applied before the sequence solve to reduce the risk that the nonlinear optimiser enters a mirrored arm configuration.

Reproducibility requires control of library interfaces as well as data files. The checked GMR revision expects an earlier positional form of the Mink inverse-kinematics API, whereas the installed Mink version assigns a different meaning to the corresponding argument. A narrow compatibility adapter maps the call explicitly and records the detected limits-API mode in every artefact. Without this adapter, the program can either fail at runtime or, more dangerously, run with constraints in the wrong parameter position. The adapter is kept at the integration boundary rather than modifying the optimisation algorithm.

4.5.3 Collision-aware retargeting

Pose matching alone can place a robot hand through its torso or allow opposite limbs to intersect. Collision avoidance is therefore enabled during trajectory generation. The project’s G1 self-collision preset defines nine groups of non-adjacent body pairs. Adjacent links are omitted because their meshes naturally overlap around a joint and would create persistent false constraints. Hand geometries that are visual-only in the source model are activated for collision evaluation.

The current preset uses a 5 mm minimum separation, an 80 mm detection distance, a gain of 0.85, and no constraint-bound relaxation. Validation uses a 6 mm separation requirement, providing a 1 mm numerical buffer beyond the generation constraint. The expanded preset currently yields 226 geometry pairs, although this count is recorded rather than assumed because a robot-model revision may change the geometry set.

Collision validity is checked against both the GMR retargeting model and the MuJoCo playback model. This dual-model check matters because different XML files can use different meshes, contact exclusions, or visual/collision roles. Where shallow contact behaviour differs between MuJoCo versions, the validator uses explicit geometry-distance queries so that the intended separation contract remains visible. The scope of this kinematic validation is stated with the complete preflight procedure in Section 4.7.1.

4.5.4 Continuity conditioning

Even a collision-free per-frame solution can contain an abrupt change between neighbouring frames. After retargeting, joint positions are first clamped for minor numerical overshoot be-

yond their model limits. Root quaternions are renormalised and their signs are made continuous. Segment velocities are then limited to

$$\|\dot{\mathbf{q}}_{\text{joint}}\|_{\infty} \leq 3\pi \text{ rad s}^{-1}, \quad \|\dot{\mathbf{p}}_{\text{root}}\|_2 \leq 3 \text{ m s}^{-1}, \quad (4.11)$$

and root angular speed is limited to $4\pi \text{ rad s}^{-1}$. For a violating segment, the correction is not accepted solely from endpoint tests. The candidate interpolation is sampled at 40 intermediate poses and checked against the collision constraints. A 14-iteration binary line search finds the largest safe progress towards the proposed frame. Two stabilisation passes are used because restricting one segment changes the starting state of the next. If the first retargeted frame is colliding, it is backed off towards the neutral configuration.

This stage conditions the source trajectory but does not close its loop. A forced last-to-first match would change valid one-shot motion and hide the real seam that the online transition controller must handle. Loop suitability and clip-to-clip blending are therefore addressed during playback.

4.6 Canonical Trajectory and Dataset Build

4.6.1 Canonical G1 artefact

Each successfully retargeted dance is stored as a canonical binary artefact. Its primary arrays are

$$\mathbf{P} \in \mathbb{R}^{N \times 3}, \quad \mathbf{Q}_r \in \mathbb{R}^{N \times 4}, \quad \mathbf{Q}_j \in \mathbb{R}^{N \times 29}, \quad (4.12)$$

representing root position, root orientation in **wxyz** order, and G1 joint positions. The stored joint-name list fixes the semantic order of all 29 columns. Array width alone is insufficient because two 29-DoF models may use a different order. The artefact also stores frame rate, format and pipeline versions, source identifier and digest, SMPL-model digest, retargeter name and full revision, collision settings, compatibility mode, and continuity limits.

4.6.2 Cached builds, quarantine, and atomic publication

Retargeting hundreds of clips is expensive, so the builder supports validated reuse. Before reuse, a cached artefact is checked for the expected schema, source and model hashes, complete GMR revision, frame rate, joint order, quaternion convention, collision preset, and continuity settings. Array shapes, finite values, quaternion norms, and sign continuity are checked as well. A file that fails validation is renamed with an `.invalid` suffix rather than being left under its canonical name.

New trajectories are generated to a temporary `.build` path. Only a fully validated output atomically replaces the canonical path. Dataset manifests follow the same publish-after-validation rule. A complete build of an official split may replace its manifest. A partial build is merged so that an interrupted experiment does not erase unrelated valid rows. Failed identifiers are removed from the active manifest and written to a separate failure report.

These measures turn the manifest into a declaration of validated availability. They also make interrupted builds recoverable. The next run can reuse verified artefacts, quarantine incompatible

ones, and continue without assuming that an existing filename is complete.

4.7 Trajectory Preflight and Catalogue Admission

4.7.1 MuJoCo kinematic preflight

The catalogue builder treats trajectory loading as a testable operation. The preflight validator first repeats the canonical schema and provenance checks. It rejects non-finite arrays, an unexpected joint order, invalid quaternion norms, missing source identity, disabled collision generation, or an abbreviated software revision. It also applies secondary corruption thresholds of 20 rad s^{-1} for joint speed, 10 m s^{-1} for root translation, and 20 rad s^{-1} for root angular speed. These are looser than the generation limits in Section 4.5.4. Their role is to detect malformed data.

The trajectory is then loaded into the headless G1 playback model and grounded. For each frame, the validator checks finite joint values and actual model joint ranges. At a fixed stride of four frames it writes the kinematic state into MuJoCo and calls forward evaluation. Generalised position, velocity, and actuator-state arrays must remain finite, and the minimum foot height must not fall below -10^{-5} m after grounding. No actuator command is applied in this test.

Preflight is therefore a kinematic admission test. It catches schema mismatch, model-order errors, invalid states, excessive discontinuities, and grounding failure at catalogue-build time. Dynamic tracking, torque limits, balance, and hardware safety require a later controller and a separate experimental evaluation.

4.7.2 Joining audio and motion records

After preflight, the builder joins each valid motion profile to its music segments. A row enters the searchable catalogue only if the audio source, source SMPL motion, canonical G1 artefact, key-pose profile, and preflight result are all present. Excluded rows are written to a dedicated CSV file with a machine-readable reason. This strict inner join prevents a high-scoring audio entry from referring to a motion that the online process cannot load.

The current catalogue contains 411 paired manifest rows. All 411 motion rows passed admission in the recorded build, with zero excluded motions. Because several dances share music, these rows refer to 60 music identifiers, 70 representative audio variants, and 348 audio segments. The global median equivalent motion speed is approximately 1.31 m s^{-1} . These figures document the current artefact. A rebuild with different data, model weights, or validation settings must update them.

4.8 Integrity Guarantees and Remaining Limitations

The finished offline catalogue provides five guarantees to the online matcher. First, the query and reference audio paths share one documented extractor and one learned-model identity, so their descriptors are compared within a consistent feature space. Second, dense matrices have known dimensions and reference-set normalisation statistics, preserving comparability across live queries. Third, every selectable motion has stable source and retargeting provenance, including source and model hashes, software revision, and generation settings. Fourth, key poses and activity statistics have been computed consistently from the source sequence, providing comparable entry

candidates and motion profiles. Fifth, the exact trajectory can be loaded, grounded, and evaluated in the target MuJoCo kinematic model, with its joint ordering, numerical validity, and model compatibility checked before admission.

Together, these guarantees establish a traceable connection between an audio retrieval result and an available robot trajectory. Validated caching, quarantine of incompatible artefacts, and atomic publication reduce the risk of exposing stale or incomplete outputs. The final audio-motion join further ensures that searchable entries refer to motions with the required profiles and preflight results. These guarantees remain specific to the recorded data, model assets, and validation settings. Changes to those dependencies require the corresponding artefacts to be rebuilt or revalidated.

But these guarantees stop short of several stronger claims. A Discogs-derived embedding is not a direct measure of choreographic correctness: musical similarity does not establish that a particular motion suits the phrasing or expressive intent of a new passage. A velocity valley is not necessarily a semantic dance boundary, and a low-velocity source pose does not by itself guarantee a smooth transition from the robot’s current state. Retargeting and continuity conditioning may also alter the motion around a source-derived key pose, so entry suitability still requires online evaluation.

IK and self-collision constraints do not prove balance, torque feasibility, or safe hardware execution. The MuJoCo preflight evaluates kinematic states without applying actuator commands, and its sampled checks do not establish continuous-time dynamic feasibility. Likewise, validating individual clips does not validate every possible blend, loop seam, or playback-speed change. Finally, AIST++ pairings reflect the choreography in the dataset and may not cover arbitrary live music or all desired G1 behaviours.

Chapter 5

Real-Time Music–Motion Matching and Continuous Control

5.1 Chapter Overview

Chapter 4 constructed a catalogue in which every audio reference has a consistent descriptor and every selectable dance has a preflight-passed Unitree G1 trajectory. This chapter develops the online method that uses this catalogue. The method receives an audio stream, retrieves musically related references, ranks their associated motions, accumulates evidence for a stable selection, and connects the selected trajectory to the currently playing one. At the same time, a causal beat controller adjusts motion phase without discontinuously jumping between poses.

The proposed method is retrieval-based rather than generative. It cannot create an unseen choreography, but it offers three practical advantages for the present humanoid system. First, the source of every commanded pose is known. Second, expensive retargeting and preflight are completed before live use. Third, retrieval, selection, timing, and transition errors can be measured separately. These properties complement recent work on expressive audio-driven humanoid locomotion and streaming character generation, which demonstrates the importance of musical responsiveness but also highlights the difficulty of causal control [19, 15].

The online method is organised around three different decisions:

- (1) *what* music and motion are supported by the recent audio.
- (2) *when* the evidence is stable enough to authorise a switch.
- (3) *how* the current and selected robot trajectories can be joined without a pose, velocity, contact, or root discontinuity.

5.2 Online Problem Formulation

Let $\mathcal{A}_{\leq t}$ denote audio observed up to current time t , and let $\mathcal{M} = \{M_i\}_{i=1}^N$ be the finite catalogue of robot-ready motions. Each M_i is associated with source music u_i , an authored tempo, an activity profile, key-pose phases, foot-contact states, and a G1 trajectory. The online system

selects motion M_{i_t} and phase ϕ_t , then produces the emitted configuration $\mathbf{q}_t^{\text{out}}$ using only $\mathcal{A}_{\leq t}$ and stored state:

$$(i_t, \phi_t, \mathbf{q}_t^{\text{out}}) = \mathcal{G}(\mathcal{A}_{\leq t}, \mathcal{M}, \mathcal{H}_t), \quad (5.1)$$

where $\mathcal{H}_t$ contains the current motion, recent rankings, accepted beats, played-motion history, prepared candidates, and controller state. The history term makes explicit that selection is stateful. Applying a memoryless nearest-neighbour rule independently to every window would make i_t oscillate when several catalogue items have similar scores.

Throughout the thesis, $\mathcal{F}$ denotes audio feature extraction and $\mathcal{G}$ denotes stateful online control. The waveform history $\mathcal{A}_{\leq t}$ is distinct from its descriptor $\mathbf{x}_t$. The full catalogue $\mathcal{D}$ contains descriptors, trajectories M_i and metadata $\mathbf{m}_i$; $\mathcal{M}$ is its trajectory collection, and $\mathcal{C}_t$ is the ranked motion candidate set. Thus $\mathbf{x}_i$ and M_i denote different parts of an item, rather than alternative names for it. The index i_t identifies the selected motion, ϕ_t is playback phase, and $\mathbf{q}_t^{\text{out}}$ is the emitted robot configuration after blending and output constraints. The time subscript refers to the current update instant; Δt is the elapsed time between updates.

The method optimises no single end-to-end objective at runtime. Instead, it uses a sequence of constrained local objectives:

$$\begin{aligned} \text{audio retrieval:} & \quad \max s_{\text{track}}, \\ \text{motion ranking:} & \quad \max s_{\text{final}}, \\ \text{entry search:} & \quad \min C_{\text{entry}}, \\ \text{playback:} & \quad \min |e_\phi| \quad \text{subject to phase-rate and output constraints.} \end{aligned} \quad (5.2)$$

This factorisation reflects the available evidence. Audio descriptors can indicate semantic and rhythmic compatibility, but they cannot determine the instantaneous joint configuration at which a switch is smooth. Conversely, joint-space entry cost cannot decide whether a dance is appropriate for the music.

5.2.1 Dual-timescale causal processing

The same audio stream is processed at two timescales. A six-second rolling waveform supplies the context required by the catalogue extractor. A retrieval attempt is made once per second when the previous attempt has completed. In parallel, a short-block analyser emits activity, onset, spectral, and beat events used for phase control and subtle pose modulation. The retrieval view therefore estimates musical identity and compatibility, whereas the event view estimates local timing.

Both paths are causal. No sample later than the current audio position is used. This is stricter than merely playing an offline model incrementally, a distinction emphasised in real-time diffusion control [15]. The longer retrieval window does introduce an observation delay at startup and after a musical change. This delay is explicit and can be measured. It is not hidden by using future context.

Feature extraction and catalogue search run in a single background worker. If that worker is occupied, the next submission opportunity is skipped. Requests are not accumulated in a queue. Thus the live controller consumes the newest completed evidence instead of processing

an increasing backlog of stale audio windows. Completed rankings are passed to the bounded candidate-preparation subsystem described in Section 5.4.3.

5.3 Hierarchical Music-to-Motion Retrieval

5.3.1 Query standardisation and segment similarity

For every query window, the online extractor produces the same three vectors as the offline pipeline: a learned embedding $\mathbf{e}_q$, a rhythm–timbre vector $\mathbf{r}_q$, and style activations $\mathbf{g}_q$. The embedding is derived from the same Discogs EffNet model used to construct the catalogue [2]. Model identity and feature dimensions are checked at startup. A mismatch is treated as an error.

Learned and rhythm vectors are standardised using the catalogue statistics and then normalised to unit length. For feature family $f \in \{e, r\}$,

$$\bar{\mathbf{f}}_q = \frac{(\mathbf{f}_q - \boldsymbol{\mu}_f) / \max(\boldsymbol{\sigma}_f, \epsilon)}{\|(\mathbf{f}_q - \boldsymbol{\mu}_f) / \max(\boldsymbol{\sigma}_f, \epsilon)\|_2}. \quad (5.3)$$

Style activations are unit-normalised without z-standardisation because their coordinates have a direct label meaning. Cosine similarity is mapped into $[0, 1]$:

$$s_f(q, n) = \frac{1 + \bar{\mathbf{f}}_q^\top \bar{\mathbf{f}}_n}{2}. \quad (5.4)$$

For catalogue segment n , the fused score is

$$s_{\text{seg}}(q, n) = 0.70s_e(q, n) + 0.20s_r(q, n) + 0.10s_g(q, n). \quad (5.5)$$

The learned representation supplies the main semantic neighbourhood, while explicit rhythm prevents two stylistically similar but rhythmically dissimilar passages from being treated as identical. Style activations have the smallest weight because they are useful priors but are too coarse to determine a track on their own. If style-vector dimensions are unavailable, the remaining weights are renormalised to 0.70/0.90 and 0.20/0.90. Missing evidence is not interpreted as zero similarity.

5.3.2 Track pooling and genre-first restriction

A track normally contributes several overlapping catalogue segments. Using its single best segment is sensitive to accidental local agreement, whereas an average over the entire track penalises longer recordings and music with legitimate internal variation. Let $s_{u,(1)} \geq s_{u,(2)} \geq \dots$ be the ordered segment scores belonging to track u . The track score is the mean of at most the three highest values:

$$s_{\text{track}}(q, u) = \frac{1}{K_u} \sum_{k=1}^{K_u} s_{u,(k)}, \quad K_u = \min(3, N_u). \quad (5.6)$$

Tracks are next grouped by their AIST++ genre code. An initial genre score is the same top-three mean over tracks in that genre. Discogs labels are also mapped to broad AIST++ families. For example, hip-hop labels support the middle- and low-hip-hop families, while house and techno

labels support the house family. These mappings are deliberately broad priors rather than a claim that Discogs and AIST++ define identical taxonomies.

When the largest mapped tag prior is at least 0.08, the final genre score is

$$s_{\text{genre}}(h) = 0.35 \tilde{s}_{\text{track-genre}}(h) + 0.65 \tilde{p}_{\text{tag}}(h), \quad (5.7)$$

where both terms are normalised within the current query. The default style-first policy admits the highest-scoring family. It also admits the second family when the genre-score margin is no more than 0.025. All later motion ranking is restricted to admitted families and to tracks among the retrieved set. This hierarchy reduces an important failure mode of flat retrieval. A small acoustic advantage from an unrelated genre cannot be compensated later by a mechanically convenient motion.

5.3.3 Abstention and match confidence

The system is allowed to abstain. It computes a negative-style score as the largest activation among non-music, ambient, drone, field-recording, noise, and related tags. If this value is at least 0.17 for two consecutive retrieval windows, the query is rejected as non-dance or ambient material. Persistence prevents one uncertain tag activation from cancelling a valid dance selection. A query is also rejected when the beat-quality value defined in Equation 4.3 is below 0.35.

For diagnostics, confidence combines beat quality q_b and the leading genre margin Δ_h :

$$c_{\text{match}} = \text{clip}_{[0,1]} \left(0.55 q_b + 0.45 \text{clip}_{[0,1]} \left(\frac{\Delta_h}{0.10} \right) \right). \quad (5.8)$$

This confidence value explains the strength of the evidence but does not override the explicit rejection rules. When a result is rejected, no new motion evidence is accumulated and the current motion remains active. Therefore, the system fails by holding a known trajectory.

5.3.4 Motion-specific compatibility

Music retrieval identifies relevant recordings, but each recording can be associated with several dances that differ in speed, event density, and activity. A second ranking stage evaluates the motion itself. Only motions whose offline preflight passed and whose music and genre survived the preceding stage are considered.

Tempo compatibility handles half-time and double-time ambiguity. For query BPM b_q and the source tempo b_i of motion M_i , the candidate playback ratios are

$$\mathcal{R}_i = \left\{ \frac{b_q}{b_i}, \frac{b_q}{2b_i}, \frac{2b_q}{b_i} \right\}. \quad (5.9)$$

Only ratios in $[0.55, 1.60]$ are retained. The valid ratio closest to one in log space is selected, and its compatibility is

$$s_{\text{tempo}}(i) = \exp \left[-\frac{1}{2} \left(\frac{\log r_i}{0.28} \right)^2 \right]. \quad (5.10)$$

If no ratio is feasible, the motion is excluded. If reliable tempo is absent, the implementation assigns a deliberately weak score of 0.25 at authored speed.

Rhythmic compatibility compares the audio event rate with the motion key-pose rate. The query and motion rates are

$$\rho_q = 0.70 \frac{b_q}{60} + 0.30 d_o, \quad \rho_i = r_i d_{k,i}, \quad (5.11)$$

where d_o is onset density and $d_{k,i}$ is confidence-weighted key-pose density. Their scale-symmetric agreement and the motion regularity term are

$$s_{\text{density}}(i) = \exp \left[-\frac{1}{2} \left(\frac{|\log(\rho_i/\rho_q)|}{0.65} \right)^2 \right], \quad (5.12)$$

$$s_{\text{key}}(i) = 0.80 s_{\text{density}}(i) + 0.20 \exp[-\min(c_{v,i}, 2)]. \quad (5.13)$$

When either rate is effectively zero, $s_{\text{key}} = 0.25$. This avoids an undefined logarithm while preserving the uncertainty penalty.

Motion activity is obtained from the catalogue-relative 90th-percentile velocity. With catalogue median μ_v and robust scale σ_v ,

$$a_i = \frac{1}{1 + \exp[-(v_{90,i} - \mu_v)/\sigma_v]}, \quad s_{\text{activity}}(i) = 1 - |a_q - a_i|, \quad (5.14)$$

where a_q is the audio activity from Equation 4.2. The complete motion and final scores are

$$s_{\text{motion}}(i) = 0.50 s_{\text{tempo}}(i) + 0.35 s_{\text{key}}(i) + 0.15 s_{\text{activity}}(i), \quad (5.15)$$

$$s_{\text{final}}(i) = 0.65 s_{\text{track}}(u_i) + 0.35 s_{\text{motion}}(i). \quad (5.16)$$

Music evidence remains dominant, but the motion-specific term prevents a semantically similar track from authorising choreography that would require extreme time scaling or has incompatible event density. All component values are retained for evaluation and ablation.

5.4 Stable Selection and Candidate Readiness

5.4.1 Relevance hysteresis

Let i_t^* be the leading motion at retrieval update t , and let i_c be the currently playing motion. A relevance switch is armed only when

$$i_t^* \neq i_c, \quad s_{\text{final}}(i_t^*) \geq s_{\text{final}}(i_c) + 0.08, \quad (5.17)$$

for three consecutive completed retrievals with the same i_t^* . If the leader changes, the current motion becomes best, the score margin disappears, or the match is rejected, the streak is reset. A successful streak creates a *pending relevance selection*. It does not immediately alter playback.

This hysteresis is important because overlapping audio windows are strongly correlated. Three wins should not be interpreted as three independent statistical observations. They are instead a temporal persistence rule that filters short score inversions while giving sustained evidence a bounded path to a switch.

5.4.2 Diversity under a relevance constraint

A stable musical passage can keep the same motion at rank one for a long time. To avoid visible repetition, diversity is handled by a second policy. It considers the latest top five motions and retains only candidates satisfying

$$\begin{aligned} s_{\text{final}}(i) &\geq s_{\text{final}}(i^*) - 0.05, \\ s_{\text{track}}(u_i) &\geq s_{\text{track}}(u_{i^*}) - 0.08. \end{aligned} \tag{5.18}$$

The candidate must remain eligible for the same three-update streak. After the current motion has been held for four accepted-beat bars, the selector first prefers a motion cluster absent from the three-item recent history, then an unseen motion identifier, and finally the least recently played eligible item. The score constraints are applied before recency, so variation cannot introduce an arbitrarily poor match.

Relevance and diversity selections record different reasons. A relevance selection can be committed at the next valid boundary as soon as it is ready. A diversity selection is committed only after the maximum hold has actually been reached. This distinction allows a genuine musical change to take priority over a cosmetic rotation.

5.4.3 Accepted-beat commitment and prepared candidates

The switch clock counts accepted alignment beats rather than raw detector peaks. Every four accepted beats defines a bar boundary. A pending selection becomes eligible for commitment at this boundary, ensuring that switching and phase control share one rhythmic interpretation. The logical state progression is

$$\text{current} \rightarrow \text{evidence} \rightarrow \text{pending} \rightarrow \text{prepared} \rightarrow \text{transitioning} \rightarrow \text{current}. \tag{5.19}$$

Whenever a ranking arrives, the pending motion and other high-ranked motions are submitted to a bounded preparation pool. Preparation includes canonical trajectory loading, grounding in the playback model, joint-name adaptation, and construction of the entry features used in Section 5.6. As a bar boundary approaches, ready candidates are scored against an estimate of the source phase expected at that boundary.

The pending identifier remains the preferred choice whenever it is fully prepared. Entry cost chooses the best phase within that motion. It does not silently replace a musically preferred motion with a less relevant one. If the preferred motion is not ready, a different ranked and prepared candidate may be used as an explicit fallback. If no evidence-backed candidate is ready, the system continues the current trajectory. File I/O and grounding are never performed synchronously merely to meet a boundary.

5.5 Causal Beat and Phase Control

5.5.1 Activity gate and beat candidates

Microphone startup includes a short calibration from which the 90th percentile of RMS and onset change estimates the local noise floor. Subsequent audio is active only when its RMS exceeds both a fixed floor and a multiple of the calibrated floor. A short release interval prevents the activity state from chattering near the threshold. When audio is inactive, beat history is reset and motion amplitude returns gradually towards zero.

Beat candidates are produced by predominant local pulse (PLP) estimation, following Grosche and Müller [11], through `librosa.beat.plp` [22, 21] on a rolling history of already acquired audio. PLP combines locally estimated periodicities into a pulse curve; its peaks supply beat candidates. Block-level onset and loudness histories supply local event contrast. Analysis can revise estimates inside its available window, but the controller acts only when results are delivered and cannot revise emitted poses. This online PLP path is distinct from the dynamic-programming beat tracker used for catalogue descriptors [8]. In the present method, each candidate has pulse confidence c_p and recent-event contrast c_x . Its selection score is

$$c_b = (1 - w_x)c_p + w_x c_x, \quad w_x = 0.5. \quad (5.20)$$

Candidates below the hard confidence threshold $c_p < 0.35$ are rejected before they can influence phase or the bar counter.

5.5.2 Motion-aware beat acceptance

Dense music may contain subdivisions faster than the current dance can travel between authored key poses. The controller therefore derives an admissible time interval from the next key-pose gap T_k and the playback-speed limits:

$$T_{\min} = \eta \frac{T_k}{r_{\max}}, \quad T_{\max} = \frac{T_k}{r_{\min}}, \quad (5.21)$$

where η is the key-point interval ratio and is one in the current matcher configuration. A beat earlier than $T_{\min}$ after the previous accepted beat is rejected. When the detector itself proposes periods shorter than this feasible interval, recent candidate scores define an adaptive quantile threshold. Weak subdivisions are rejected early in the admissible window. The threshold falls as elapsed time approaches $T_{\max}$. At the deadline, an eligible beat is accepted so that the controller cannot wait indefinitely for an ideal accent.

This policy treats beat detection and beat use as different problems. The analyser may report all plausible pulses, but only events compatible with the motion’s next authored interval become control targets. Beat-synchronised dance methods likewise demonstrate that temporal correspondence must be represented explicitly rather than inferred from visual plausibility alone [14].

5.5.3 Target rate and gradual phase correction

For accepted beat index k , the controller selects the corresponding authored key-pose phase ϕ_k^* . The wrapped phase error is

$$e_\phi = \text{wrap}_{[-1/2, 1/2)}(\phi_k^* - \phi). \quad (5.22)$$

The first accepted beat uses the full error as a correction request. Later events use gain $0.25 + 0.35c_b$, so a high-quality beat creates a stronger but still gradual correction. In adaptive mode, if $\Delta\phi_k$ is the authored phase gap and Δt_k is the last accepted-beat interval, the target phase rate is

$$\omega^* = \text{clip}_{[r_{\min}\omega_a, r_{\max}\omega_a]} \left(\frac{\Delta\phi_k}{\Delta t_k} \right), \quad (5.23)$$

where ω_a is the authored phase rate. The running base rate is exponentially smoothed towards ω^* . Phase correction is not applied by assigning $\phi \leftarrow \phi_k^*$. Instead, the remaining correction e_r is consumed with time constant τ_p :

$$\Delta\phi_c = e_r \left(1 - e^{-\Delta t / \tau_p} \right). \quad (5.24)$$

The correction is clipped so that the effective phase rate remains within the same speed interval. Its change is also slew-limited. With the resulting ω_{eff} , phase evolves continuously as

$$\phi_{t+1} = \text{wrap}_{[0, 1)}(\phi_t + \omega_{\text{eff}}\Delta t). \quad (5.25)$$

This mechanism synchronises by temporarily accelerating or decelerating the authored motion, not by teleporting to a different frame. When beat evidence times out, the target rate returns to ω_a . When a new motion controller is created, it inherits the previous tempo, amplitude, beat history, and effective-rate state, while its phase is initialised at the selected entry. Thus switching does not restart rhythmic estimation.

5.5.4 Bounded pose modulation

Timing adaptation preserves the selected trajectory as the primary source of movement. A secondary modulation layer adds small, zero-centred offsets from low-, mid-, and high-frequency energy, rhythm density, off-beat ratio, and beat accent. In the default subtle mode, the authored pose is not multiplied by a global amplitude. Instead, bounded offsets are applied mainly to the waist, shoulders, wrists, and knees. For example, high-frequency activity produces a small alternating wrist and shoulder detail, while low-frequency energy produces a small knee component.

This layer is intentionally secondary. It improves immediate responsiveness between slower retrieval updates without changing motion identity or root trajectory. Because it can create poses not present in the offline artefact, its result still passes through the output conditioning described in Section 5.8.

5.6 Transition-Entry Optimisation

5.6.1 Entry candidates and state features

Motion matching and motion-graph methods establish the value of comparing state features at possible transition points [16, 13]. The present method uses this principle only after music has supplied a small candidate set. For each target key pose, the search includes its central frame and up to three frames on either side. The neighbouring candidates receive linearly decreasing salience. This local expansion can find a materially closer robot configuration without turning the entry search into an unconstrained scan over the whole clip.

At each candidate, the cached state contains named joint positions and velocities, joint ranges, grounded root position, root tilt after removing yaw, root linear velocity, root angular speed, and binary left/right foot contacts. Only joint names shared by source, target, and the output-limits table are used. The current source phase is mapped to its nearest source frame.

5.6.2 Normalised entry cost

For source frame s and target candidate j , joint-pose and velocity costs are

$$C_q(j) = \sqrt{\frac{1}{D} \sum_{d=1}^D \left(\frac{q_{j,d} - q_{s,d}}{R_d} \right)^2}, \quad (5.26)$$

$$C_v(j) = \sqrt{\frac{1}{D} \sum_{d=1}^D \left(\frac{\dot{q}_{j,d} - \dot{q}_{s,d}}{v_d^{\max}} \right)^2}, \quad (5.27)$$

where R_d is the model joint range and $v_d^{\max}$ is the configured speed limit. Contact cost is the fraction of the two feet whose support states differ:

$$C_c(j) = \frac{1}{2} \sum_{f \in \{L, R\}} \mathbf{1}[c_{j,f} \neq c_{s,f}]. \quad (5.28)$$

Root cost averages four separately normalised disagreements:

$$C_r(j) = \frac{1}{4} \left(\frac{|z_j - z_s|}{0.15} + \frac{|\theta_j - \theta_s|}{0.35} + \frac{\|\dot{\mathbf{p}}_j - \dot{\mathbf{p}}_s\|_2}{3.0} + \frac{|\omega_{r,j} - \omega_{r,s}|}{4\pi} \right), \quad (5.29)$$

where θ is non-yaw root tilt. The key-pose term $C_m(j) = 1 - s_j$ favours a salient velocity valley. The total entry cost is

$$C_{\text{entry}}(j) = 0.45C_q(j) + 0.20C_v(j) + 0.20C_c(j) + 0.10C_r(j) + 0.05C_m(j). \quad (5.30)$$

Pose mismatch receives the largest weight because it is the most immediately visible discontinuity. Velocity and contact distinguish frames that have a similar pose but incompatible

movement or support. Root state and musical salience refine the decision without overriding a severe whole-body mismatch.

5.6.3 Feasibility and transition duration

An entry with low instantaneous cost can still be unsuitable if it occurs near the end of a one-shot target. The conservative remaining playback time is

$$T_{\text{remain}}(j) = \frac{(1 - \phi_j)T_i}{r_{\text{max}}}. \quad (5.31)$$

It must cover both the transition and at least one remaining bar under the default configuration. The unquantised transition lower bound is derived from joint displacement and the same dynamics limits used by the output layer:

$$T_{\text{raw}}(j) = \max_d \left(\frac{1.5|q_{j,d} - q_{s,d}|}{v_d^{\text{max}}}, \sqrt{\frac{6|q_{j,d} - q_{s,d}|}{a_d^{\text{max}}}} \right). \quad (5.32)$$

After enforcing a minimum, the duration is rounded upward to a half-beat multiple:

$$T_{\text{tr}}(j) = \text{clip}_{[0.35, 1.20]} \left(\left\lceil \frac{\max(0.35, T_{\text{raw}}(j))}{T_b/2} \right\rceil \frac{T_b}{2} \right). \quad (5.33)$$

The candidate is eligible when $T_{\text{remain}} \geq T_{\text{tr}} + T_{\text{bar}}$. Candidates whose raw requirement exceeds the maximum transition duration are avoided whenever a feasible alternative exists. Eligible candidates are ordered first by total cost, then by salience cost, remaining duration, and phase. If none leaves the required duration, the earliest key-pose candidate is used as a controlled fallback. This fallback is recorded. It is not presented as an optimal entry.

5.7 Continuous Transition and Root Motion

5.7.1 Target-root alignment

Catalogue trajectories use clip-local roots. Before a target is blended, its entry root is aligned to the current world state in horizontal position and yaw. Let $(\mathbf{p}_s, R_s)$ be the target clip’s entry root and $(\mathbf{p}_c, R_c)$ the current root. With $Y(R)$ denoting the yaw-only component,

$$A = Y(R_c)Y(R_s)^{-1}. \quad (5.34)$$

For every target frame $(\mathbf{p}, R)$, the aligned horizontal root and orientation are

$$\mathbf{p}'_{xy} = \mathbf{p}_{c,xy} + [A(\mathbf{p} - \mathbf{p}_s)]_{xy}, \quad (5.35)$$

$$R' = AR. \quad (5.36)$$

The grounded vertical coordinate and the target roll and pitch are preserved. The same rigid transform is retained throughout the transition, so target displacement is preserved rather than repeatedly forcing its root onto the source.

5.7.2 Smooth pose and orientation blending

For elapsed transition fraction $u \in [0, 1]$, the blend weight is the cubic smoothstep

$$h(u) = 3u^2 - 2u^3. \quad (5.37)$$

Joint positions and root translation use linear interpolation with $h(u)$,

$$\mathbf{q}(u) = (1 - h(u))\mathbf{q}_e(u) + h(u)\mathbf{q}_n(u), \quad (5.38)$$

while root orientation uses shortest-path quaternion spherical interpolation. Both the old and new motion controllers continue to advance during the blend. Consequently, the transition connects two moving trajectories rather than freezing their entry frames.

After blending, a persistent root-continuity transform maps the active clip-local displacement into one world trajectory. On a source change, the world anchor becomes the last emitted world root and the new clip origin becomes its current raw root. In continuous mode,

$$\mathbf{p}_{w,xy}(t) = \mathbf{a}_{xy} + [A_a(\mathbf{p}(t) - \mathbf{p}_0)]_{xy}, \quad (5.39)$$

where $\mathbf{a}$ and A_a are the stored world anchor. This prevents each new catalogue clip from returning the robot to the dataset origin.

5.7.3 One-shot completion and state transfer

The catalogue motions are treated as one-shot trajectories. Phase is clamped before it can sample the implicit last-to-first interpolation used by generic preview samplers. When a clip approaches its end, a prepared transition can be started even if no regular bar switch is ready. Its blend progress is coupled to the remaining source phase so that the old trajectory is fully released before the terminal pose stalls. If no prepared, ranked candidate exists, the last pose is held. An optional very small stationary breathing signal can avoid a perfectly frozen appearance without moving the root.

At blend completion, the target sampler, phase controller, phase tracker, root references, and motion identifier are committed together. Only after this transfer does the selection policy reset its held-bar count and record the new item in recency history. This avoids a mixed frame in which, for example, the new identifier is paired with the old controller state.

5.8 Output Conditioning

5.8.1 Stateful joint dynamics limits

Entry optimisation and smoothstep blending reduce discontinuity, but music modulation and two simultaneously moving clips can still produce an excessive joint target increment. A stateful limiter therefore operates on the final joint target q_d . For every joint d , the emitted velocity and acceleration must satisfy

$$|\dot{q}_d| \leq v_d^{\max}, \quad |\dot{q}_d - \dot{q}_d^-| \leq a_d^{\max} \Delta t. \quad (5.40)$$

The limiter computes the velocity needed to approach q_d , clips it by speed, and then clips its change around the previous emitted velocity. When a target stops, a braking-distance test begins deceleration early enough to avoid overshoot. If the target can be reached within both limits, it is snapped only to the feasible target rather than passing beyond it. Per-joint limits may be loaded from a validated table. Otherwise configured defaults are used. Limiter activation, requested speed, emitted speed, and emitted acceleration are retained as diagnostics.

5.8.2 Collision policy and state consistency

Offline preflight covers each canonical trajectory, but it cannot certify every pose created by online blending or modulation. The runtime supports collision checking for modified poses or for every pose. When enabled, collision handling is applied after the joint limiter because it must inspect the pose that would otherwise be emitted. If it changes that pose, the limiter is reset to the actual collision-resolved output. Without this reset, the next update would compute acceleration from a state that the robot never received.

The current default runtime collision mode is disabled because full geometric checking has a nontrivial online cost. Experiments that claim online collision protection must therefore enable and report the selected policy explicitly. Regardless of this runtime option, all source motions have passed the offline preflight described in Chapter 4. The resulting pose is finally written by joint name into the G1 MuJoCo model for kinematic state evaluation [25].

5.9 Method Observability and Failure Behaviour

The online trace follows the processing boundaries established above. Retrieval records acceptance, rejection reason, genre, score margins, component scores, BPM, and leading tags. Selection and preparation record pending reason, recent cluster, ready candidates, load failures, and explicit fallback use. Control and transition records contain accepted-beat time, phase rate, remaining phase correction, entry-cost components, transition duration, blend progress, root deltas, limiter intervention, and collision override.

These fields distinguish failures that can look alike in the final animation. For example, a late accent can be attributed to beat rejection, phase-rate saturation, transition blending, or the joint limiter. Repetition can be attributed to weak retrieval evidence, rejection, readiness, or the diversity clock. The associated fallback is always conservative: the system skips stale work, excludes failed candidates, and otherwise retains the last valid active state. It never responds to failure with an arbitrary catalogue item or synchronous retargeting. The trace supplies the intermediate measurements used by the experiments in the next chapter.

Chapter 6

Experimental Evaluation

6.1 Experimental Questions and Protocol

This chapter evaluates BeatWeaver as a retrieval-based music-to-motion system with continuous kinematic playback. It separates the quality of the selected motion from the quality and timeliness of its execution. A fast catalogue search does not establish a fast musical response, and a bounded intermediate joint command does not establish that the final displayed pose satisfies the same bounds.

Five research questions organise the evaluation:

1. RQ1: does the catalogue support meaningful retrieval for unseen music, and can unsuitable inputs be rejected?
2. RQ2: does beat-synchronised timing improve rhythm alignment, and does the selection policy maintain diversity without losing relevance?
3. RQ3: are motion transitions and final outputs continuous and within the specified kinematic and clearance constraints?
4. RQ4: which computational stages dominate latency, and does the complete system meet its real-time operating targets?
5. RQ5: which component effects are supported by ablations and long-duration operation?

6.2 Datasets, Splits and Reproducibility

The local catalogue contains 60 music identities, 348 descriptor segments and 411 selectable motions spanning ten AIST++ genres. These are the assets available to this implementation, not the complete AIST++ dataset [18]. Strict leave-one-music-out evaluation removes every descriptor, track record and motion associated with the query music identity before ranking. The 60 music identities define the independent units of this retrieval evaluation.

The offline motion-distribution subset freezes four music identities per genre. Each identity is paired with its lexicographically first preflight-passed ground-truth motion. Five seeds yield 200

Evaluation	Inputs and conditions	Seeds	Count
Offline retrieval	60 held-out music identities	–	60
Offline rejection	30 recordings; overlapping windows	–	750
Offline SMPL features	40 music/reference pairs; reconstructed output	0–4	200
120 Hz pilot, 60 s	3 inputs; full and authored	0–2	18
120 Hz pilot, 306 s	1 stitched input; full	0–2	3
60 Hz formal, 40 s	13 inputs; full	0–4	65
60 Hz formal, 40 s	Same 13 inputs; authored	0–4	65
60 Hz formal, 606 s	1 stitched input; full	0–4	5

Table 6.1: Evaluation matrix. Counts are queries, windows, reconstructed outputs, or runtime runs as indicated. Runtime durations include six seconds of warm-up. Only the final three rows constitute the 135 formal runs.

reconstructed outputs, evaluated as five separate sets of 40 against the same 40 references. These outputs use preanalysed musical timing and trace-based SMPL reconstruction; they characterise motion features and do not measure online beat delivery or control deadlines. The evaluation excludes six seconds of warm-up and reconstructs the following 20 seconds at 60 frames per second. The deployed catalogue is used without a leave-one-music-out exclusion, so this is not another held-out retrieval test.

The causal short suite compares the full system with authored timing on the same 13 inputs: the stitched AIST++ recording, ten CC0 recordings, a tempo jump, and a silence/recovery stream. Each condition has 65 runs of 40 seconds, including six seconds of warm-up. Five full-system long runs use 606-second inputs, giving approximately 600 seconds of post-warm-up observation per seed. Short source recordings are deterministically repeated when necessary. In particular, the stitched source has a 39-second cycle because of its crossfades. Its repetition boundary is itself a musical change, not a seamless continuation.

The rejection evaluation includes the ten CC0 inputs and 20 synthetic negative inputs, with 750 overlapping query windows in total. The negatives comprise silence, pink noise, ambient drones and speech-like signals. They constitute a synthetic stress test, not coverage of recorded human speech or environmental sound. The window-level rejection scores below are descriptive. Adjacent windows are not treated as 750 independent recordings.

Table 6.1 states the experimental units and run counts. The 135 formal runs are exactly $13 \times 2 \times 5 + 1 \times 1 \times 5$. The separate 120 Hz capacity pilot comprises 18 short runs: three inputs (stitched AIST++, silence/recovery, and a 90-to-150 BPM tempo jump), two conditions (full and authored timing), and seeds 0–2. Each uses 60 seconds of input, including six seconds of warm-up. Three full-system runs of the repeated stitched input use the same seeds and 306 seconds each, including warm-up. The 21-run denominator excludes the ten-second smoke check.

The runtime experiments are serial and headless, with a six-second matching window, one-second matching interval, collision checking and final-output auditing enabled. The pilot tests the 120 Hz design target stated in Section 1.3: control-work p99 below 8.33 ms and a deadline-miss ratio below 1% for each run. Only six of its 21 runs pass all gates; the worst p99 is 15.572 ms and the worst miss ratio is 0.18544. The formal operating point was consequently set to 60 Hz before the formal matrix was executed. Its p99 budget is 16.67 ms; the 1% miss threshold and final-output constraints remain the same. This is a reduced operating target, not evidence that the

initial 120 Hz requirement was met. Different source coverage and durations also prevent treating the pilot and formal matrix as a paired estimate of the effect of frequency. Their results are not pooled. The recorded platform has 20 logical CPU cores and an RTX 3080, but ONNX inference uses `CPUExecutionProvider` exclusively. Python 3.11.14, NumPy 1.26.4, librosa 0.10.2.post1 [21], ONNX Runtime 1.28.0, SciPy 1.11.4 and MuJoCo 3.10.0 are recorded in the manifest.

Every formal run preserves its command, source identity, seed, trace, timing report, final-pose archive and log. The audit checks the declared run matrix, artifact hashes, strictly increasing output clocks and consistent robot identity. All 538 frozen input-file hashes matched during the post-run audit, and the formal run index contained no failed attempts. Source-file hashes supplement the Git commit identifier to capture uncommitted implementation changes. Pilot and formal artifacts have separate manifests.

6.3 Evaluation Metrics

6.3.1 Retrieval, rejection and statistical units

Genre Recall@ K , macro- F_1 , mean reciprocal rank (MRR) and NDCG@5 measure catalogue matching. Source motion recall is retained only as an internal same-catalogue sanity check. It cannot establish generalisation because the target and candidates share catalogue provenance. CC0 genre labels are sets of acceptable catalogue genres, rather than a complete taxonomy of the recordings.

For runtime quality metrics, frames are first reduced to a run summary. Seeds are then averaged within each source recording. Confidence intervals use 2,000 bootstrap resamples of source means. Ablations use two-sided paired sign-flip permutation tests after averaging matched seeds within each source. Holm correction covers the reported condition-by-metric tests within each suite. The direction of a mean difference alone is not evidence of a component benefit. For the long-run suite, five seeds of one repeated source quantify repeatability. A one-source bootstrap interval is degenerate and must not be read as strong generalisation evidence.

6.3.2 Rhythm and motion diversity

Following the beat-alignment evaluation family used in dance generation [18, 24], this chapter defines the music-to-dance score as

$$A_{m \rightarrow d} = \frac{1}{|B_m|} \sum_{t \in B_m} \exp\left(-\frac{\min_{u \in B_d} (u - t)^2}{2\sigma^2}\right), \quad \sigma = \frac{3}{60} \text{ s.} \quad (6.1)$$

The reverse score is calculated by exchanging the beat sets. Their harmonic mean is the main BAS summary. Robot output is resampled to 60 FPS. Motion beats are local minima of mean body-speed magnitude, with a minimum separation of 0.20 seconds. The stationary MuJoCo world body is excluded and the pelvis is used as the positional reference.

For causal runs, B_m contains the actual delivery times of all online detected beats, not retrospectively estimated beat times and not only the subset accepted by the phase controller. Consequently the score assesses alignment to delivered control information, not accuracy against an annotated musical beat ground truth. When no music beats are available, BAS is undefined and excluded from numerical averages with its missingness retained.

Fréchet Inception Distance (FID) compares Gaussian approximations to reference and output feature distributions [12]. Following the AIST++/FACT and Bailando evaluation convention [18, 24], this chapter uses motion descriptors in place of image Inception features. The subscripts k and g identify kinetic and geometric feature spaces: the pinned AIST++ extractor returns 72 kinetic components (horizontal and vertical kinetic energy and an acceleration-based expenditure statistic for each of 24 joints) and 32 geometric components summarising body-configuration predicates. The extractor is fixed to `google/aistplusplus_api` commit `2dd7b3e946b794fd0081c98e2e2433545abf8b87`; local changes concern imports and formatting only.

For feature space $h \in \{k, g\}$, let $\boldsymbol{\mu}_{R,h}$ and $\boldsymbol{\Sigma}_{R,h}$ be the reference sample mean and covariance, and let $\boldsymbol{\mu}_{O,h}$ and $\boldsymbol{\Sigma}_{O,h}$ be their output counterparts. The reported distance is

$$\begin{aligned} \text{FID}_h = & \|\boldsymbol{\mu}_{R,h} - \boldsymbol{\mu}_{O,h}\|_2^2 \\ & + \text{tr}\left(\boldsymbol{\Sigma}_{R,h} + \boldsymbol{\Sigma}_{O,h} - 2\left(\boldsymbol{\Sigma}_{R,h}^{1/2}\boldsymbol{\Sigma}_{O,h}\boldsymbol{\Sigma}_{R,h}^{1/2}\right)^{1/2}\right). \end{aligned} \quad (6.2)$$

Lower FID indicates closer feature distributions, not necessarily better musical correspondence. Diversity (Div) is the mean Euclidean distance among all distinct output feature-vector pairs, in the same evaluation family [24]:

$$\text{Div}_h = \frac{2}{n(n-1)} \sum_{a < b} \|\mathbf{z}_{a,h} - \mathbf{z}_{b,h}\|_2, \quad n = 40. \quad (6.3)$$

Here $\mathbf{z}_{a,h}$ is the descriptor of output a in feature space h . Each seed uses all $\binom{40}{2} = 780$ unordered pairs, with the reference Div calculated identically on the 40 ground-truth items. Larger Div means greater spread; it is interpreted relative to reference spread rather than maximised without regard to plausibility.

Both metrics use reconstructed AIST++/SMPL motion, not G1 joint angles. Raw results use the extractor’s native scale. Standardised results transform both sets using reference means and population standard deviations, with each denominator floored at 10^{-8} . FID itself uses sample covariances. The kinetic covariance is necessarily rank deficient with only 40 samples in 72 dimensions; the implementation uses a symmetric eigendecomposition with negative numerical eigenvalues clipped to zero. Finite-sample FID can be biased [7], so neither this stabilisation nor averaging five seeds establishes comparability with larger published benchmarks.

Robot-domain diversity additionally reports unique selected motions, catalogue-cluster coverage and normalised selection entropy. This entropy is computed over *selection episodes*, after collapsing consecutive identical motion IDs. It is not a dwell-time-weighted entropy. Catalogue clusters are engineered motion descriptors, not human judgements of visual style.

6.3.3 Continuity, contact and constraints

Final joint positions are read from MuJoCo qpos after limiting, collision projection and joint-range clipping. Velocity uses successive actual wall-clock differences. Acceleration and jerk use the corresponding midpoint clocks.

The safety audit distinguishes candidate clearance detections, corrective projections and vi-

Retrieval metric	Value	Rejection metric	Value
Recall@1	0.317	Precision	0.964
Recall@3	0.533	Recall	0.720
Recall@5	0.783	F_1	0.824
Macro- F_1	0.272	AUROC	0.646
MRR	0.505	False rejection	0.062
NDCG@5	0.550	Missed rejection	0.280

Table 6.2: Strict leave-one-music-out genre retrieval (60 music identities) and descriptive window-level rejection (750 windows from 30 recordings)

Genre	N	Recall@1	Recall@5
BR	6	0.333	1.000
HO	6	1.000	1.000
JB	6	0.000	0.833
JS	6	0.000	0.500
KR	6	0.000	0.500
LH	6	0.167	0.667
LO	6	0.500	1.000
MH	6	0.667	0.833
PO	6	0.333	1.000
WA	6	0.167	0.500

Table 6.3: Per-genre leave-one-music-out results. Each genre has six music identities. Codes: BR = breaking; HO = house; JB = ballet jazz; JS = street jazz; KR = krump; LH = LA-style hip-hop; LO = locking; MH = middle hip-hop; PO = popping; WA = waacking [27, 18]

olations remaining in the final pose. The configured clearance target is 6 mm, comprising the existing 5 mm minimum plus a 1 mm buffer. A residual clearance violation therefore does not, by itself, prove geometric penetration. Conversely, a correction count is not a residual violation count. Both are useful but answer different questions.

Physical Foot Contact (PFC) was introduced to assess dance plausibility [26]. The G1 adaptation follows the official evaluator’s algebra on a 30 FPS grid. The magnitude of root acceleration, after clamping its downward component, is normalised by its sequence maximum and multiplied by the horizontal displacements of the two feet. The result is averaged and scaled by 10^4 . Unlike the SMPL implementation’s ankle/toe minima, G1 uses one recorded body anchor per foot. Zero root acceleration is assigned zero score instead of an undefined division. These explicit differences preclude direct numerical comparison with published SMPL PFC results. Foot skating and support-height diagnostics are also retained in the machine-readable results.

6.4 Retrieval and Rejection Results

Recall@1 is 0.317, whereas Recall@5 reaches 0.783 (Table 6.2). Relevant genre information is therefore more reliably retained in a shortlist than selected as the single best answer. The results support a useful retrieval signal, but not consistently correct genre identification for previously unseen music.

The per-genre results in Table 6.3 prevent the overall average from obscuring genre-dependent failure. The rejection AUROC is 0.646. The operating-point precision and recall must be con-

CC0 input	Genre hit rate	Rejection rate
01_electronic_backbeat	1.000	0.000
02_hiphop_hippety_hop	0.040	0.400
03_jazz_bebop_for_joey	1.000	0.000
04_world_bollywood_groove	0.840	0.000
05_latin_cumbish	1.000	0.000
06_upbeat_bar_brawl	0.000	0.000
07_orchestral_battle_ready	0.200	0.000
08_waltz_isolation_waltz	0.000	0.160
09_funk_funky_energy_loop	0.000	0.000
10_ambient_alien_spaceship	–	0.960

Table 6.4: Per-recording CC0 results from offline query evaluation

Condition	N	Harmonic BAS [95% CI]	G1 PFC [95% CI]
Causal full	65	0.313 [0.270, 0.348]	4.139 [2.882, 5.766]
Causal authored	65	0.306 [0.269, 0.340]	5.001 [2.866, 7.911]
Causal long	5	0.335 [0.335, 0.335]	1.456 [1.456, 1.456]

Table 6.5: Source-averaged rhythm and adapted physical-contact scores. (N is the number of runs)

sidered alongside that ranking score and the unequal number of positive and negative windows. Precision alone does not establish robust rejection.

Table 6.4 reports each CC0 recording rather than treating ten heterogeneous inputs as a large domain-generalisation benchmark. Together, the retrieval and rejection results answer RQ1 partially: the catalogue provides useful candidates, but neither top-one matching nor rejection is sufficiently reliable to serve as an unqualified deployment guarantee.

6.5 Rhythm, Diversity and Physical Plausibility

Table 6.5 and Figure 6.1 expose variability that would be hidden by reporting a single BAS. The valid timing ablation is full versus authored within the causal short suite: sources, seeds, input lengths and measurement instruments are matched. The offline reconstructed feature subset supplies a separate description of motion spread and is not included in this timing comparison.

In the matched causal comparison, full-system harmonic BAS is 0.313, compared with 0.306 for authored timing. The full-minus-authored gain is 0.0067 with a source-bootstrap 95% interval of $[-0.0078, 0.0228]$. The interval includes zero and the Holm-adjusted permutation p -value is 1.0000. The formal causal evidence therefore does not establish that beat-synchronised timing improves BAS relative to authored timing under this delivered-beat metric.

Table 6.6 is deliberately a within-protocol characterisation, not a leaderboard comparison.

Scale	FID_k	FID_g	Div_k	Div_g
raw	$2.93e+07 \pm 2.63e+07$	0.643 ± 0.031	3214.397 ± 841.277	0.675 ± 0.008
real standardised	73.438 ± 28.920	$1.40e+12 \pm 8.04e+11$	4.504 ± 0.854	$4.77e+05 \pm 1.47e+05$

Scale	Ground-truth Div_k	Ground-truth Div_g
raw	4089.854	1.441
real standardised	9.239	7.326

Table 6.6: Reconstructed SMPL feature metrics: mean and standard deviation over five separate 40-output seed evaluations.

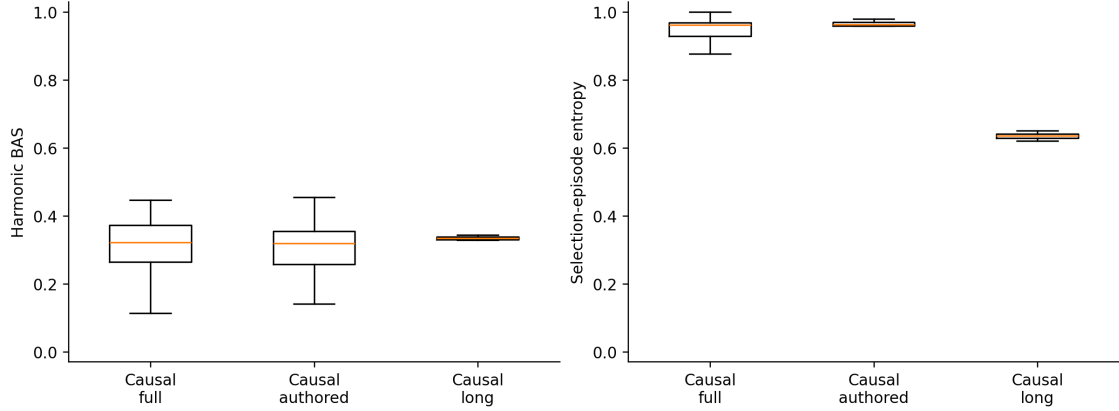


Figure 6.1: Distributions of run-level BAS and selection-episode entropy across the three formal conditions.

Trace phases and blend factors are interpolated within unchanged motion/transition segments, while categorical switch boundaries are handled separately. Reconstruction aligns root translation but cannot recover every sub-trace boundary or the robot’s final yaw, limiting and projection effects. Short ground-truth motions are cycled, potentially introducing root seams. Finally, the system retrieves real source motions, whereas an unconstrained generator must produce its own trajectories. These differences can bias FID and Div, and diversity is interpreted relative to the reference distribution.

The raw geometric Div is only 0.468 of the ground-truth value, indicating a narrower reconstructed geometric feature distribution. More importantly, some ground-truth geometric dimensions have zero variance. Dividing output deviations by the implemented 10^{-8} standard-deviation floor produces the extremely large standardised geometric values in Table 6.6. Those entries diagnose ill-conditioned normalisation, not a meaningful ranking of dance quality. They are not used as a primary success criterion. The large raw kinetic scale and its seed variability must also be read alongside the cyclic-source and reconstruction limitations, rather than compared directly with published generation benchmarks.

Figure 6.2 reports catalogue-cluster coverage per input alongside the per-run maximum final-output speed. Coverage complements entropy: an approximately uniform distribution over a small visited subset is not broad catalogue exploration. Likewise, low adapted PFC does not establish smooth transitions or balanced locomotion. These measures jointly qualify the answer to RQ2 instead of reducing dance quality to one score.

6.6 Transition Continuity and Safety

The causal short full-system peak is 15.4 rad/s and the long-run peak is 15.4 rad/s. These are final-output, actual-clock derivatives, not the limiter’s internal nominal-step report. Table 6.7 therefore evaluates a stricter and more relevant output contract than checking the limiter in isolation. The logged position limits, clearance outcomes and dynamic bounds must each be judged separately.

All 135 causal formal runs pass the registered final-output contract at 60 Hz. The maximum final speed is 15.4 rad/s for the causal short full system, 15.4 rad/s for authored timing and 15.4

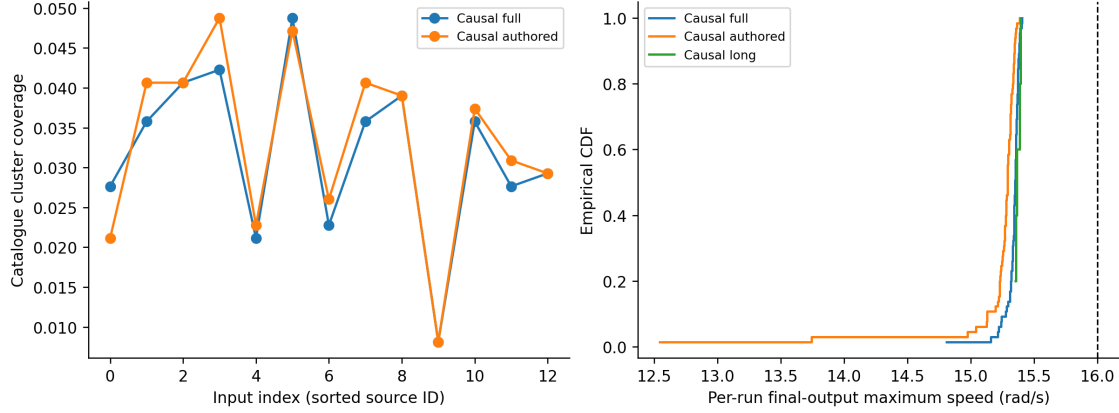


Figure 6.2: Left: mean catalogue-cluster coverage across five seeds per causal short-suite input, ordered by source ID. Right: empirical distributions of per-run maximum final-output speed. The dashed line is the 16 rad/s target.

Causal condition	N	Max speed	Max accel.	Residual frames	Limit frames
Causal full	65	15.4	1673.4	0	0
Causal authored	65	15.4	1523.3	0	0
Causal long	5	15.4	1633.8	0	0

Table 6.7: Post-warm-up final-output diagnostics

rad/s for the long runs, in every case below the 16 rad/s limit; the largest acceleration reported in Table 6.7 is 1673.4 rad/s², below the 2000 rad/s² limit. Residual-clearance and final joint-range violation counts are both zero. Candidate collision detections and corrective projections still occur, so this result establishes the audited final-pose contract rather than an absence of interventions.

Figure 6.3 shows the corresponding behaviour around a single switch, where the transition is the point at which entry error, blending and the output limiter interact most strongly. The output pipeline measures actual output intervals, constrains collision projection by the remaining dynamic envelope and audits the emitted MuJoCo state. The formal result supports RQ3 for the registered kinematic bounds at 60 Hz.

Condition	Query p95	Work p99	Miss ratio	p99 fails	Miss fails
Causal full	50.5	4.66	0.000	0/65	0/65
Causal authored	50.6	4.77	0.000	0/65	0/65
Causal long	116.2	8.72	0.000	0/5	0/5

Table 6.8: Medians across runs of each run’s query p95, control-work p99 and deadline-miss ratio. Times are in ms. Failure counts apply the 16.67 ms p99 and 1% deadline-miss targets to individual runs. These are not pooled frame percentiles.

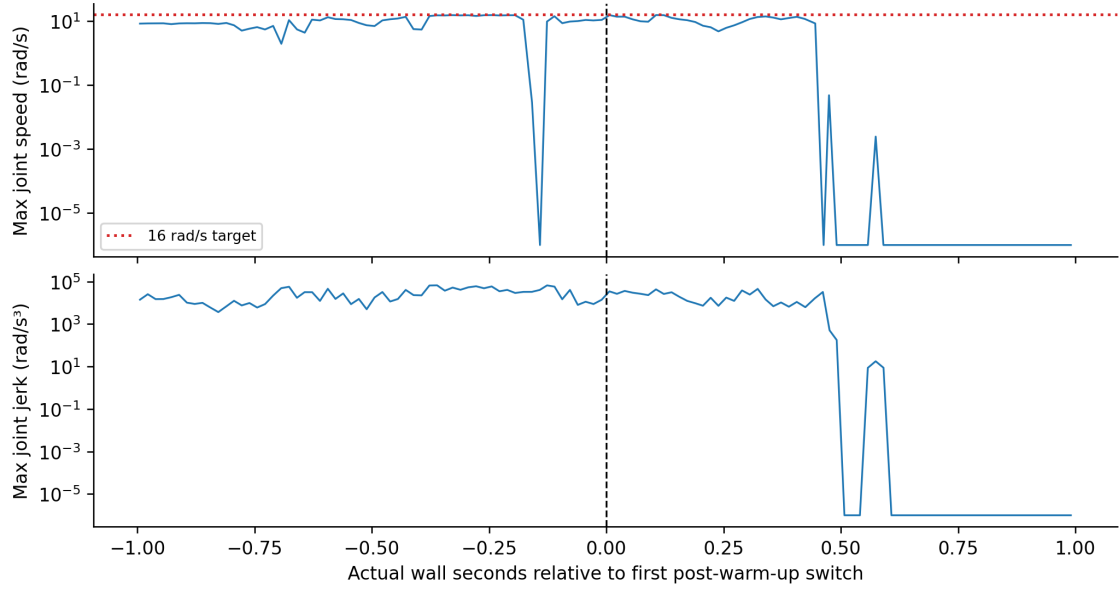


Figure 6.3: Actual-clock speed and jerk around the first post-warm-up switch in the fixed causal-full stitched seed-0 run

6.7 Inference Time and End-to-End Response

6.7.1 Computation, preparation and control work

The causal short full-system median run-level query p95 is 50.489 ms, but its corresponding control-work p99 is 4.657 ms (Table 6.8). Retrieval latency and control deadline compliance are therefore separate outcomes. The long-run median p99 is 8.724 ms and its median deadline-miss ratio is 0.000. Figure 6.4 gives the full per-run distributions behind these medians, so that the margin to each target can be read rather than inferred from a central value. No formal 60 Hz run fails either real-time gate. The worst individual long run remains below the registered limits, at 14.483 ms p99 and 0.00693 deadline-miss ratio.

Tables 6.9 and 6.10 distinguish ONNX execution, hand-crafted features, track retrieval, motion ranking, candidate preparation and control work. Candidate grounding and readiness are background operations. Their wall-clock preparation latency is not an additional synchronous delay on every control frame. However, asynchronous execution does not remove resource contention or the need for a candidate to be ready when selected.

The recorded ready-pool underrun and hold-last totals are both zero across all 135 causal formal runs (Table 6.11). The prepared-cache fallback therefore meets the registered operational readiness targets in this cohort.

The final-output audit performs an additional clearance check, including forward kinematics. Its overhead is reported separately and remains inside total control work. First-call/cold labels in some stage counters also do not establish an OS-cache-cold benchmark. Complete cold/warm distributions for every stage, and a separate mel-construction timer, are not uniformly available. The chapter reports the timing actually recorded.

Submission attempts require particular care. While a retrieval worker is busy, the control

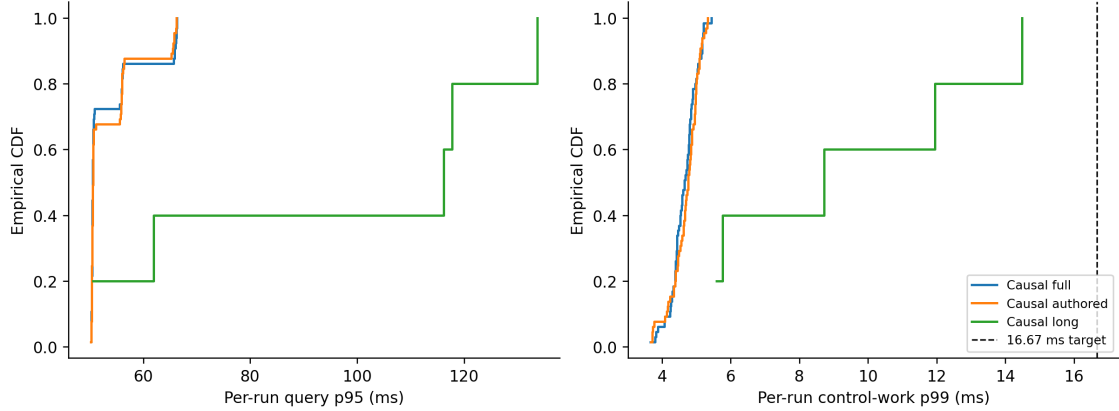


Figure 6.4: Empirical CDFs of per-run query p95 and control-work p99, with the 16.67 ms control-work target marked.

loop can attempt submission repeatedly before a successful update advances the matching clock. Consequently, busy attempts divided by all attempts are not automatically a one-second query-slot loss rate. A completed/submitted ratio can also include one query still in flight at shutdown. The machine-readable operational counters are retained, but a 99% scheduled-slot completion claim is not established from those counters alone.

6.7.2 Musical changes and observable response

Figure 6.5 separates a new genre-compatible ranking from pending selection, switch initiation, half blending and completion. Six seconds of accumulated context are needed for the first query, and a post-change window contains a mixture of old and new audio until that context has been replaced. Query scheduling, repeated-winner evidence, bar boundaries and candidate readiness can introduce further waiting. The six-second window is therefore neither an ONNX inference time nor a universal fixed response latency.

Milestone traces are censored at the next musical change, including the repeated stitched boundary in long runs. Genre-target matching is available for stitched inputs. Tempo-jump and silence/recovery milestones are scheduling diagnostics, not independently verified correct-tempo or correct-rejection responses. Pending and switch events also need not refer to an identical candidate throughout a complex change episode. These limitations prevent interpreting every observed milestone chain as a successful semantic response. RQ4 is answered at the stage and control-loop level, while fully attributed end-to-end responsiveness remains a stricter future evaluation requirement.

6.8 Ablation Study

The formal ablation compares full beat-synchronised control with authored timing on the same 13 inputs and five matched seeds. Authored timing disables beat-driven retiming while retaining retrieval, motion selection, transitions, collision checking and output constraints. The formal evaluation does not isolate the contributions of retrieval scoring, diversity selection, entry search

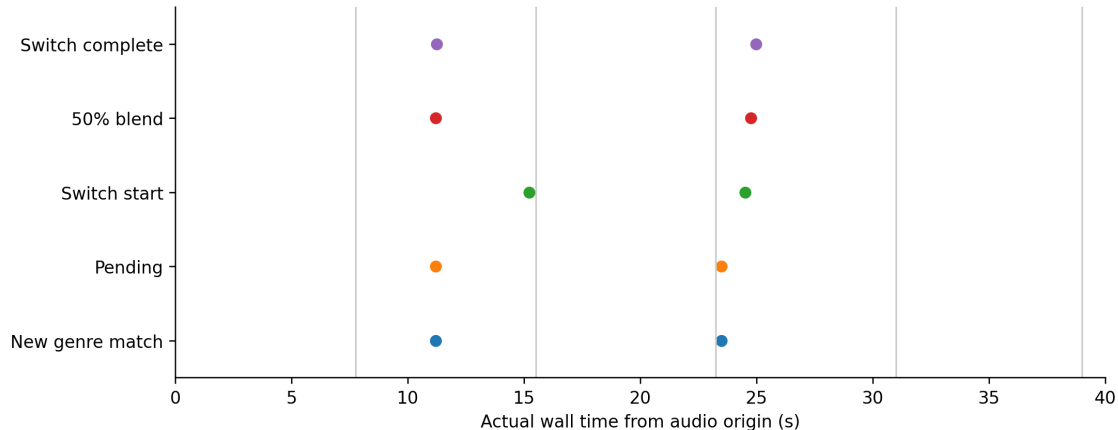


Figure 6.5: Response milestones after each musical change in the causal-full stitched AIST++ run at seed 0. Grey vertical lines mark the change times; the five series give the first genre-compatible match, the pending selection, switch initiation, half blending and switch completion. The run is selected deterministically as the lexically first stitched source at seed 0, not chosen for a favourable response profile.

or output limiting; those component benefits remain unestablished.

Table 6.12 reports paired timing comparisons after averaging matched seeds within each source. The 65 source/seed combinations are not treated as 65 independent musical inputs. The displayed tests retain Holm correction over the full set of recorded condition-by-metric tests in the short suite, rather than recomputing correction over only the displayed rows. There is no corrected evidence of a BAS difference. Absence of significance is not proof of equivalence. The source-mean deadline-miss ratio differs from full by 0.0000 (adjusted $p = 1.0000$), and both conditions pass the absolute 60 Hz gates. RQ5 is therefore answered for the timing condition and sustained operation, with the other component effects left open. Conditions were executed serially in a fixed matrix order, rather than randomised across time. Background-load drift remains a possible timing confound. Here “causal” describes audio availability, not a guarantee of unconfounded causal inference about every performance difference.

6.9 Long-Run Reliability

All five causal long runs completed their registered duration, passed artifact integrity and passed every evaluated real-time, readiness and final-output gate. Repeating a 39-second source for approximately ten minutes stresses recurrent switching and resource use, but supplies only one source distribution. The causal long runs provide sustained evidence for the selected 60 Hz operating point. The separate 120 Hz pilot has 15/21 failures, so the initial design target remains unmet on this platform.

6.10 Discussion by Research Question

For RQ1, the leave-one-music-out shortlist provides stronger evidence than the same-source motion sanity check, but top-one and rejection errors remain material. For RQ2, rhythm alignment is measurable under a genuinely online delivery protocol, yet it remains a proxy for musical timing and does not replace judged dance quality. Coverage, entropy and reference-distribution diversity describe complementary properties and should not be collapsed into an unrestricted diversity objective.

For RQ3, auditing final qpos verifies the registered end-to-end kinematic constraints at 60 Hz while preserving the distinction from dynamics and hardware safety. For RQ4, retrieval computation and asynchronous preparation are explicitly measurable, and the selected 60 Hz operating point passes the empirical control gates; the 120 Hz target does not. For RQ5, the paired statistics delimit the timing claim; the other component contributions have not been isolated by this formal matrix.

Overall, BeatWeaver is an auditable retrieval and playback pipeline that meets its registered kinematic and empirical real-time contracts at 60 Hz on the measured platform. It is not a hard-real-time or physically safe humanoid dance controller.

Stage (causal full short)	p50	p95	p99	Worst max
Audio copy	0.070	0.090	0.122	0.678
Audio preprocessing	1.914	2.204	2.391	3.565
Hand-crafted features	24.924	25.915	26.311	29.382
ONNX inference	15.541	16.350	16.765	20.175
Track retrieval	2.169	2.353	2.424	3.462
Motion ranking	0.375	0.470	0.528	1.009
Selection policy	0.023	0.033	0.052	0.126
Waveform to match	49.927	50.489	60.867	67.213
Motion file load	13.686	17.908	17.936	63.979
Grounding	257.758	375.522	390.527	1334.635
Entry features	16.845	22.297	22.385	83.780
Entry scoring	0.333	0.947	1.403	2.782
Submission to ready	1025.265	1861.576	2026.973	8383.264
Audio/beat control stage	0.041	0.546	0.714	4.368
Phase controller	0.018	0.043	0.060	0.689
Pose sampling	0.103	0.144	0.203	1.642
Transition/root blending	0.163	0.368	0.548	3.063
Limiter/collision	0.718	2.793	3.416	8.235
MuJoCo forward	0.214	0.309	0.434	2.573
Final-output safety audit	0.073	0.132	0.318	2.321

Table 6.9: Stage-resolved timing for the causal full short suite

Startup stage	Median ms
ONNX session	89.8
Catalogue	21.3
Matcher initialisation	1.9
MuJoCo initialisation	134.3
Initial grounding	1362.1
Until control loop	1.04e+04

Table 6.10: Process-start measurements for the causal full short suite

Causal condition	Underruns	Runs with hold	Hold events
Causal full	0	0	0
Causal authored	0	0	0
Causal long	0	0	0

Table 6.11: Whole-run operational readiness counters

Ablation minus full	Sources	Δ	p	p_{Holm}
Causal authored: BAS	13	-0.007	0.4205	1.0000
Causal authored: miss ratio	13	0.000	1.0000	1.0000

Table 6.12: Selected paired ablation comparisons after source-level seed averaging. (Δ is ablation minus full)

Chapter 7

Conclusions and Future Work

This dissertation investigated how streaming music can guide the selection, timing and transition of humanoid dance drawn from a finite motion library. The resulting BeatWeaver connects offline AIST++ preparation and G1 retargeting with online music retrieval, stateful motion selection and continuous kinematic playback. The experiments establish a working, inspectable integration with measurable rhythm alignment and a validated 60 Hz operating point, while leaving the 120 Hz design target in Section 1.3 unmet. This chapter summarises the work, states the limits of those conclusions and identifies the developments needed for broader musical coverage and physical robot execution.

7.1 Summary of the Work

Data preparation and motion representation. The offline pipeline established a traceable association between music, human choreography and target-robot trajectories. Audio windowing, robust amplitude normalisation and source-motion analysis produced comparable music descriptors and velocity-valley key poses. SMPL sequences were converted through GMR into named-joint, floating-base trajectories for the 29-DoF Unitree G1. Versioned artefacts, source and model hashes, validated caching and MuJoCo preflight made catalogue admission explicit. The recorded build contains 411 admitted motions linked to 60 music identities and 348 audio segments. These checks establish representation and sampled kinematic validity for the stored clips. They do not certify every subsequent retiming or blend.

Hierarchical music retrieval. The final catalogue uses the Discogs EffNet ONNX representation, comprising a 1,280-dimensional embedding and 400 style activations, together with a 24-dimensional DSP rhythm-timbre descriptor. The retargeting artefacts use pipeline version 4. The DSP-only embedding remains an alternative backend, requiring a separately built catalogue, rather than the representation evaluated as the final system. Online queries use six seconds of accumulated audio and the same extraction and normalisation contract as the reference catalogue. Segment evidence is pooled at track level, filtered by genre and abstention rules, and passed to motion ranking. Strict leave-one-music-out evaluation in Chapter 6 obtained genre Recall@1 of 0.317 and Recall@5 of 0.783. Thus the representation supplies useful shortlist evidence, although

its leading result is frequently incorrect for unseen music.

Motion compatibility and stable selection. The system separates musical similarity from suitability of the associated movement. Tempo feasibility, key-pose density and motion activity refine the track-based ranking before selection. Consecutive-winner evidence and a score margin provide relevance hysteresis, while a bounded-hold policy permits variation among similarly relevant motions. Prepared-candidate requirements separate selection intent from execution readiness. These mechanisms make repetition, rejection and switching decisions observable. Their implementation is a contribution of the work, but the formal experiment isolates only the full versus authored-timing comparison. It does not establish a separate benefit for retrieval scoring, selection policy or entry search. In particular, a diversity benefit cannot be inferred merely from the presence of a rotation policy.

Beat-aware timing and gradual phase control. The timing controller distinguishes detected pulses from events accepted for motion alignment. Relative beat contrast and authored key-pose spacing guide beat acceptance, and bounded speed adaptation consumes phase error gradually instead of resetting the sampler instantaneously. In the matched causal short suite, harmonic beat alignment score (BAS) was 0.306 under authored timing and 0.313 with the full controller. The source-averaged gain was 0.0067, with a 95% bootstrap interval of $[-0.0078, 0.0228]$ and a Holm-adjusted p -value of 1.0000. The formal experiment therefore does not establish a timing benefit under the specified delivered-beat metric.

Continuous switching and output conditioning. The transition mechanism searches neighbourhoods of salient target poses using joint position, velocity, foot-contact, root-state and salience costs. Root alignment, smoothstep blending, quaternion interpolation and atomic state transfer connect successive moving trajectories. One-shot completion avoids an implicit last-to-first seam, while joint limiting and configurable collision handling condition the output. These mechanisms provide an explicit route from a selected motion to a displayed pose. The final-output audit verifies that all 135 formal causal runs satisfy the registered joint-range, speed, acceleration and clearance constraints at 60 Hz. This remains a kinematic contract rather than evidence of dynamic balance or hardware safety.

Real-time implementation and evaluation. Retrieval and candidate preparation execute in bounded background workers, while the foreground loop consumes completed results and advances playback. The evaluation records rankings, beat delivery, preparation, transitions, final poses and actual output clocks. The formal matrix in Table 6.1 contains 130 matched short runs and five long runs. All 135 pass the registered real-time, readiness and final- output gates at 60 Hz. A separate 120 Hz pilot passes only 6/21 runs, so the original frequency is not a reliable operating point on this platform. The overall achievement is therefore a reproducible retrieval-and-playback system with an empirically validated 60 Hz configuration and a measured capacity limit.

7.2 Limitations

Kinematic playback and physical feasibility. The current player writes root pose and joint positions directly to MuJoCo state and evaluates forward kinematics. It does not apply a torque-controlled tracking policy or solve closed-loop balance. Grounding and binary contact features describe geometric support, without enforcing contact forces, friction, actuator capacity or disturbance recovery. Consequently, a displayed G1 trajectory may remain dynamically infeasible even when its joint positions and sampled geometry are valid. No physical Unitree G1 deployment has been evaluated in this dissertation.

Operating-point and final-output limits. The formal 60 Hz cohort passes all registered output constraints. Maximum speed is 15.404 rad/s, maximum acceleration is 1673.415 rad/s², and both residual clearance and final joint-range violation counts are zero. Ready-pool underruns and hold-last events are also zero. These results apply to audited kinematic poses on the recorded machine. They do not establish torque feasibility, balance, contact stability or physical-robot safety.

For the causal full short suite, the median run-level control-work p99 is 4.657 ms. The five long runs have a median of 8.724 ms. All 135 formal runs remain below the 16.67 ms p99 limit and 0.01 deadline-miss-ratio limit, although the worst long run reaches 14.483 ms and 0.00693 respectively. In contrast, the separate 120 Hz pilot passes only 6/21 runs. Its worst p99 is 15.572 ms against an 8.33 ms period and its worst miss ratio is 0.18544. The 60 Hz evidence is therefore empirical soft-real-time evidence for this platform, not a hard- real-time guarantee or evidence that the original 120 Hz target has been met.

Finite coverage and unequal genre performance. The matcher adapts existing clips and cannot supply a movement vocabulary absent from the catalogue. Its coverage depends on the available AIST++ pairs, their retargeting quality and manually designed compatibility weights. Musical similarity is also an indirect proxy for choreographic suitability. Genre Recall@1 ranged from 1.000 for house (HO) to 0.000 for ballet jazz (JB), street jazz (JS) and krump (KR), using the genre key in Table 6.3, in the six-identities-per-genre evaluation. Broad Discogs-to-AIST++ label mappings and early genre restriction can propagate a classification error into the motion candidate set. These results reveal substantial class differences, while the small number of identities per class limits generalisation.

Metre assumptions and weak-beat behaviour. The switch clock groups every four accepted alignment beats into a logical bar. It does not estimate musical metre or identify downbeats. Since beat acceptance depends on motion feasibility, rejected subdivisions can also change the relationship between this clock and the underlying musical pulse. The rule is therefore poorly suited to triple metre, including waltz, and cannot promise phrase-aligned switching even for all quadruple-metre music. The observed waltz retrieval errors should not, however, be attributed to this scheduling rule without an experiment that isolates metre from retrieval.

For ambient or otherwise weakly rhythmic input, persistent negative-style and beat-quality gates provide conservative abstention. The ambient CC0 recording was rejected in 96% of evaluated windows, but one recording is insufficient evidence of robust handling of the category. Holding

the active motion or returning towards authored timing can remain musically inappropriate when the audio becomes quiet or loses its pulse. In particular, reducing secondary pose modulation does not necessarily stop the underlying authored trajectory. The system lacks a fully specified transition from dancing to low-activity behaviour and back, calibrated across diverse weak-beat recordings.

Engineering proxies and experimental coverage. No user study assesses naturalness, expressiveness or preference. BAS measures alignment to delivered detector events, while contact proxies, entropy, coverage and smoothness each capture only part of dance quality. Reconstructed SMPL FID/Div additionally depend on trace interpolation, source-motion reuse and cyclic reference construction. Near-zero reference variance makes some standardised geometric results ill-conditioned, so they cannot support an aesthetic ranking. The ten external CC0 recordings and synthetic negatives offer limited acoustic diversity. Headless file-input trials do not reproduce microphone reverberation, clipping or room noise. Repeated seeds improve repeatability evidence without creating independent musical sources, and the causal long runs repeat only one stitched source. The formal ablation covers full versus authored timing; causal component tests for retrieval, diversity and transition entry remain future work.

7.3 Future Work

Extend the validated operating envelope. The immediate systems priority is to preserve the coherent final-output contract while reducing foreground work enough to evaluate 120 Hz again. A short-horizon constrained trajectory update could incorporate collision correction within the same temporal problem as joint limiting. Its feasibility and fallback behaviour should continue to be evaluated from actual output timestamps rather than from an intermediate limiter report. Profiling should guide reductions in repeated grounding work and especially foreground collision cost, with prepared data reused under an explicit model-and-version contract. New comparisons should freeze the machine, backend, window settings and audit configuration, and counterbalance run order to reduce background-load confounding. Explicit query-slot identifiers and a persistent candidate identifier across match, pending, readiness and blend events would also make end-to-end response attribution more reliable.

Metre and downbeat-aware control. A causal beat, downbeat and metre estimator could replace the fixed four-event clock with an estimate of musical position. The scheduler should distinguish the underlying beat sequence from the subset used for motion alignment, so skipping an infeasible accent does not redefine the bar. Uncertain metre should lead to a documented beat-level fallback rather than an assumed downbeat. Evaluation should include triple metre, compound metre, tempo changes and weak percussion, using independently annotated beat and downbeat times alongside detector delivery latency. A complementary phase study should cover at least three distinct motions per genre, stratified by tempo and key-pose density. Multiple windows or seeds of one motion should not count as additional motions.

Learned transition-entry scoring. The current weighted entry cost offers a useful baseline for learning which candidate state produces a successful transition. Training examples could combine current and target pose, velocity, contact, root motion and musical context with measured transition outcomes and human pairwise preferences. A learned ranker could then capture interactions that a fixed weighted sum misses, while explicit feasibility checks continue to determine admissibility. Evaluation should hold out source motions and music identities and compare the ranker with the existing cost on both perceived continuity and final-output constraints. An improved prediction score alone would not establish a better executed transition.

Motion-fragment retrieval and composition. The existing system can enter partway through a selected clip, but retrieval and activity metadata remain primarily clip-level. A fragment catalogue could instead index phrases or contact-consistent movement units, each with local music descriptors, entry and exit states, duration and feasible tempo range. Searching compatible fragment sequences would permit finer responses to changes within a song and reuse a larger combination of existing motifs. A sparse transition graph or bounded-horizon search could balance music relevance, contact compatibility and motif continuity. Such an extension needs new checks for the joins and composed sequence. Validating each fragment separately would not establish that their concatenation is feasible. Diversity should remain subordinate to coherent musical phrasing.

Contact-constrained tracking and balance. Foot-contact states should become constraints on execution rather than only terms in entry ranking. A subsequent controller could combine support-phase information, foot placement and floating-base state estimation with whole-body inverse dynamics, model-predictive control or a learned tracking policy. Its role would be to track the selected reference subject to contact and actuator constraints, reducing motion intensity or delaying a transition when necessary. Dynamics-based simulation should evaluate tracking error, foot slip, support transitions, torque saturation and recovery from disturbances. This would test a different and stronger claim than the present kinematic preview.

Physical Unitree G1 deployment. Hardware integration should follow validated closed-loop simulation and a measured final-output contract. The music matcher would supply motion references to the tracking controller rather than directly assign the physical root state. Initial trials could progress from supported, low-amplitude movements to freestanding sequences and then music-driven transitions. They should record microphone acquisition, communication and actuator-response delays separately, as well as tracking failures and interventions. This staged evaluation would reveal how acoustic conditions, state-estimation error and actuator limits change the behaviour observed in headless file-input experiments.

User preference and weak-beat adaptation. A preference-aware selector could allow users to favour particular styles, energy levels or degrees of repetition within the feasible candidate set. Pairwise judgements of musical fit, transition naturalness and overall appeal would support both preference learning and a direct evaluation of dance quality. Weak-beat behaviour should similarly become a selectable, explicit response. For example, a controlled change to a low-activity motion followed by a gradual return when rhythmic evidence recovers. Additional real ambient,

speech, noise and mixed recordings are needed to calibrate rejection and recovery separately. User preferences should influence the choice among feasible motions while preserving execution constraints, and subjective gains should be reported alongside relevance, continuity and timing measurements.

The work provides a basis for these extensions by making the chain from musical evidence to robot pose traceable. Its current results support finite-library retrieval and a 60 Hz kinematic operating point, but do not establish that causal phase control improves BAS or that 120 Hz is reliable. They demonstrate why musical responsiveness, output feasibility and real-time compliance must be evaluated separately. Resolving their interaction is the next step towards a humanoid that can perform continuously and expressively under physical control.

Bibliography

- [1] Simon Alexanderson, Rajmund Nagy, Jonas Beskow, and Gustav Eje Henter. Listen, denoise, action! audio-driven motion synthesis with diffusion models. *ACM Transactions on Graphics (TOG)*, 42(4):1–20, 2023.
- [2] Pablo Alonso-Jiménez, Xavier Serra, and Dmitry Bogdanov. Music representation learning based on editorial metadata from discogs. In *Rao P, Murthy H, Srinivasamurthy A, Bittner R, Caro Repetto R, Goto M, Serra X, Miron M, editors. Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR 2022); 2022 Dec 4-8; Bengaluru, India.[Canada]: International Society for Music Information Retrieval; 2022. p. 825-33.* International Society for Music Information Retrieval (ISMIR), 2022.
- [3] Joao Pedro Araujo, Yanjie Ze, Pei Xu, Jiajun Wu, and C Karen Liu. Retargeting matters: General motion retargeting for humanoid motion tracking. *arXiv preprint arXiv:2510.02252*, 2025.
- [4] Juan Pablo Bello, Laurent Daudet, Samer Abdallah, Chris Duxbury, Mike Davies, and Mark B. Sandler. A tutorial on onset detection in music signals. *IEEE Transactions on Speech and Audio Processing*, 13(5):1035–1047, 2005.
- [5] Sebastian Böck, Filip Korzeniewski, Jan Schlüter, Florian Krebs, and Gerhard Widmer. madmom: A new Python audio and music signal processing library. In *Proceedings of the 24th ACM International Conference on Multimedia*, pages 1174–1178, 2016.
- [6] Dmitry Bogdanov, Nicolas Wack, Emilia Gómez Gutiérrez, Sankalp Gulati, Perfecto Herrera Boyer, Oscar Mayor, Gerard Roma Trepas, Justin Salamon, José Ricardo Zapata González, and Xavier Serra. Essentia: An audio analysis library for music information retrieval. In *Britto A, Gouyon F, Dixon S, editors. 14th Conference of the International Society for Music Information Retrieval (ISMIR); 2013 Nov 4-8; Curitiba, Brazil.[place unknown]: ISMIR; 2013. p. 493-8.* International Society for Music Information Retrieval (ISMIR), 2013.
- [7] Min Jin Chong and David Forsyth. Effectively unbiased FID and inception score and where to find them. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6070–6079, 2020.
- [8] Daniel PW Ellis. Beat tracking by dynamic programming. *Journal of New Music Research*, 36(1):51–60, 2007.

- [9] Derry FitzGerald. Harmonic/percussive separation using median filtering. In *Proceedings of the 13th International Conference on Digital Audio Effects (DAFx-10)*, Graz, Austria, 2010.
- [10] Michael Gleicher. Retargeting motion to new characters. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, pages 33–42, 1998.
- [11] Peter Grosche and Meinard Müller. Extracting predominant local pulse information from music recordings. *IEEE Transactions on Audio, Speech, and Language Processing*, 19(6):1688–1701, 2011.
- [12] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [13] Daniel Holden, Oussama Kanoun, Maksym Perepichka, and Tiberiu Popa. Learned motion matching. *ACM Transactions on Graphics (ToG)*, 39(4):53–1, 2020.
- [14] Zikai Huang, Xuemiao Xu, Cheng Xu, Huaidong Zhang, Chenxi Zheng, Jing Qin, and Shengfeng He. Beat-it: Beat-synchronized multi-condition 3d dance generation. In *European conference on computer vision*, pages 273–290. Springer, 2024.
- [15] Kaiyang Ji, Bingsheng Qian, Binghuan Wu, Kangyi Chen, Ye Shi, and Jingya Wang. Discoforcing: A unified framework for real-time audio-driven character control with diffusion forcing. *arXiv preprint arXiv:2605.28491*, 2026.
- [16] Lucas Kovar, Michael Gleicher, and Frédéric Pighin. Motion graphs. *ACM Transactions on Graphics*, 21(3):473–482, 2002.
- [17] Ronghui Li, Junfan Zhao, Yachao Zhang, Mingyang Su, Zeping Ren, Han Zhang, Yansong Tang, and Xiu Li. Finedance: A fine-grained choreography dataset for 3d full body dance generation. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10200–10209. IEEE, 2023.
- [18] Ruilong Li, Shan Yang, David A Ross, and Angjoo Kanazawa. AI choreographer: Music conditioned 3D dance generation with AIST++. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 13381–13392. Ieee, 2021.
- [19] Zhe Li, Cheng Chi, Yangyang Wei, Boan Zhu, Tao Huang, Zhenguo Sun, Yibo Peng, Pengwei Wang, Zhongyuan Wang, Fangzhou Liu, et al. Do you have freestyle? expressive humanoid locomotion via audio control. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 956–965, 2026.
- [20] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J Black. SMPL: A skinned multi-person linear model. *ACM Transactions on Graphics*, 34(6):248:1–248:16, 2015.
- [21] Brian McFee, Matt McVicar, Daniel Faronbi, Iran Roman, Matan Gover, Stefan Balke, et al. librosa/librosa: 0.10.2.post1. Zenodo, software release, 2024. Version 0.10.2.post1. <https://doi.org/10.5281/zenodo.11192913>.

- [22] Brian McFee, Colin Raffel, Dawen Liang, Daniel P. W. Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. librosa: Audio and music signal analysis in Python. In *Proceedings of the 14th Python in Science Conference*, pages 18–24, 2015.
- [23] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics*, 37(4):143:1–143:14, 2018.
- [24] Li Siyao, Weijiang Yu, Tianpei Gu, Chunze Lin, Quan Wang, Chen Qian, Chen Change Loy, and Ziwei Liu. Bailando: 3d dance generation by actor-critic gpt with choreographic memory. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11050–11059, 2022.
- [25] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.
- [26] Jonathan Tseng, Rodrigo Castellon, and Karen Liu. Edge: Editable dance generation from music. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 448–458, 2023.
- [27] Shuhei Tsuchida, Satoru Fukayama, Masahiro Hamasaki, and Masataka Goto. AIST dance video database: Multi-genre, multi-dancer, and multi-camera database for dance information processing. In *Proceedings of the 20th International Society for Music Information Retrieval Conference*, pages 501–510, 2019.

Appendix A

Source Code Repository

The source code for the BeatWeaver system described in this dissertation is available in the following GitHub repository:

<https://github.com/EddieOuyang2004/MusicRobot/tree/main>

The humanoid implementation is located in `realtime/humanoid_robot/`. Readers should consult the repository's top-level README and the README in that directory for the project structure and instructions for running the system.